

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA - CEFET/RJ *CAMPUS* PETRÓPOLIS
CURSO: BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

Modelo Matemático para a Grade de Horários Aplicado ao Curso de Engenharia de Computação do Cefet/RJ

Gabrielle Muniz Santos

PETRÓPOLIS
2025

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA - CEFET/RJ *CAMPUS* PETRÓPOLIS
CURSO: BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

Modelo Matemático para a Grade de Horários Aplicado ao Curso de Engenharia de Computação do Cefet/RJ

Gabrielle Muniz Santos

Trabalho de Conclusão de Curso apresentado ao CEFET/RJ - *campus* Petrópolis, como parte dos requisitos para obtenção do título de Bacharel em Engenharia de Computação.

Orientadora: Prof.ª Laura Silva de Assis

PETRÓPOLIS
2025

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio eletrônico ou convencional, para fins de estudo e pesquisa, desde que citada a fonte.

Cefet/RJ – Sistema de Bibliotecas / Uned Petrópolis

S237m Santos, Gabrielle Muniz.

Modelo Matemático para a Grade de Horários Aplicado ao Curso de Engenharia de Computação do Cefet/RJ / Gabrielle Muniz Santos. - 2025.

48f.: il.

Trabalho de conclusão de curso (Curso de Bacharelado em Engenharia de Computação) – Centro Federal de Educação Tecnológica Celso Suckow da Fonseca, Petrópolis (RJ), 2025.

Bibliografia: p. 41 - 42.
Orientadora: Prof.ª Laura Silva de Assis, D.Sc.

1. Programação Linear. 2. Programação linear inteira. 3. Otimização Combinatória. 4. Grade de Horários. 6. Curso de Engenharia de Computação – CEFET/RJ campus Petrópolis (RJ). 7. Monografia. I. Assis, Laura Silva de. (Orienta.). II. Título.

CDD 519.72

Elaborada por Luciana de Souza Castro CRB7/ 4812

CEFET/RJ
campus Petrópolis

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA - CEFET/RJ *CAMPUS* PETRÓPOLIS
CURSO: BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

FOLHA DE APROVAÇÃO

Modelo Matemático para a Grade de Horários Aplicado ao Curso de Engenharia de Computação do Cefet/RJ

Gabrielle Muniz Santos

Trabalho de Conclusão de Curso apresentado ao CEFET/RJ - *campus* Petrópolis, como parte dos requisitos para obtenção do título de Bacharel em Engenharia de Computação.

Orientadora: Prof.ª Laura Silva de Assis

Aprovado por:

LAURA SILVA DE ASSIS:06571252619
Assinado de forma digital por LAURA SILVA DE ASSIS:06571252619
Dados: 2025.08.08 06:33:08 -03'00'

Prof.ª Laura Silva de Assis, D.Sc. (Orientadora)

LUIS DOMINGUES TOME JARDIM TARRATACA:70567312100
Digitally signed by LUIS DOMINGUES TOME JARDIM TARRATACA:70567312100
Date: 2025.08.07 18:02:12 +01'00'

Prof. Luis Domingues Tome Jardim Tarrataca, Ph.D.

gov.br
Documento assinado digitalmente
PEDRO HENRIQUE GASPARETTO LUGAO
Data: 07/08/2025 12:04:00-0300
Verifique em https://validar.iti.gov.br

Prof. Pedro Henrique Gasparetto Lugão, D.Sc.

Julho de 2025

DEDICATÓRIA

Dedico este trabalho a Deus, pois sem Ele nada me seria possível. Aos meus pais, Moises e Valéria, e ao meu irmão Moises Junior, por toda abdicação, apoio incondicional e por nunca terem deixado de acreditar e confiar em mim, até quando eu mesma não o fazia. Aos meus amigos por todo renovo emocional nos momentos difíceis.

AGRADECIMENTO

Agradeço à minha orientadora Laura, pelo suporte e tempo dedicado para que este trabalho fosse realizado. Ao meu professor e coordenador do curso, Luis Tarrataca, por todo ensinamento e orientação. Aos professores do Cefet/RJ - *Campus* Petrópolis, por me proporcionarem o conhecimento necessário para o desenvolvimento deste trabalho.

RESUMO

O problema de agendamento de horários consiste na atribuição de atividades, como aulas, provas ou reuniões, a determinados períodos de tempo. No contexto educacional envolve a alocação de disciplinas, professores e turmas a horários da semana letiva. É um problema complexo e recorrente em instituições de ensino, demandando grande esforço para atender de forma simultânea a várias restrições pedagógicas e operacionais. Este trabalho propõe um modelo de programação linear inteira para geração automática de grades horárias, e com fins de validação e aplicação prática, o modelo foi implementado considerando especificidades do curso de Engenharia de Computação do Centro Federal de Educação Tecnológica Celso Suckow da Fonseca (CEFET) - *Campus* Petrópolis. Para avaliar o modelo, foram realizados testes com dados fornecidos pela coordenação do curso, utilizando o *solver IBM ILOG CPLEX*, e com objetivo de facilitar o futuro uso do modelo foi desenvolvido um *back-end* em *Python* que se integra a um *front-end* implementado com *TypeScript* juntamente do *framework React*. O modelo considera restrições obrigatórias, como carga horária e não sobreposição de disciplinas, além de restrições flexíveis como a preferência dos docentes e conflitos entre disciplinas com alta taxa de reprovação. Com entrada estruturada em *JSON*, o modelo permite rápida adaptabilidade à mudanças e, aplicado a dados reais, foi capaz de gerar horários ótimos em cerca de 3 minutos.

Palavras-chave: Programação Linear Inteira, Grade de Horários, Otimização Combinatória.

ABSTRACT

The timetabling problem consists of assigning activities, such as classes, exams, or meetings, to specific time periods. In the educational context, it involves allocating courses, teachers, and student groups to time slots during the academic week. This is a complex and recurring issue in educational institutions, requiring significant effort to simultaneously satisfy multiple pedagogical and operational constraints. This work proposes an integer linear programming model for the automatic generation of academic timetables. For validation and practical application, the model was implemented considering the specific requirements of the Computer Engineering program at the Centro Federal de Educação Tecnológica Celso Suckow da Fonseca (CEFET) - *Campus* Petrópolis. To evaluate the model, tests were conducted using real data provided by the program's coordination, employing the IBM ILOG CPLEX solver. In order to facilitate future use, a Python-based back-end was developed and integrated with a front-end implemented using TypeScript and the React framework. The model incorporates hard constraints, such as teaching load and non-overlapping classes, as well as soft constraints, such as teachers preferences and conflicts between courses with high failure rates. With structured input in JSON format, the model allows for rapid adaptation to changes and, when applied to real data, was able to generate optimal timetables in approximately three minutes.

Key-words: Integer Linear Programming, Timetabling Problem, Combinatorial Optimization.

LISTA DE FIGURAS

1 Região viável de um modelo de PLI. 4
2 Matriz Curricular - Engenharia de Computação. 6
3 Tela inicial do sistema. 29
4 Upload do arquivo com dados de entrada do modelo. 30
5 Inserção direta de dados. 30
6 Resumo geral dos dados carregados. 31
7 Disciplinas e informações. 31
8 Preferências dos dias letivos de professores. 32
9 Mensagem de erro devido a arquivo *JSON* com dados obrigatórios ausentes. 33
10 Grade horária com filtro semanal. 34
11 Grade horária com filtro diário. 34
12 Grade horária - arquivo *PDF*. 35
13 Fluxograma do sistema de geração de grade horária otimizada. 36
14 Grade horária do período 1. 44
15 Grade horária do período 2. 44
16 Grade horária do período 3. 45
17 Grade horária do período 4. 45
18 Grade horária do período 5. 46
19 Grade horária do período 6. 46
20 Grade horária do período 7. 47
21 Grade horária do período 8. 47
22 Grade horária do período 9. 48
23 Grade horária do período 10. 48

LISTA DE TABELAS

1 Comparação entre diferentes abordagens para o problema de grade de horários. . . . 12
2 Pesos atribuídos às restrições fracas nos experimentos. 37
3 Resumo das penalidades associadas às restrições fracas com 10 matérias críticas. . . 38
4 Resumo das penalidades associadas às restrições fracas com 13 matérias críticas. . . 38
5 Resumo das penalidades associadas às restrições fracas com 15 matérias críticas. . . 38

Lista de Siglas

API	-	*Application Programming Interface*
CEFET	-	Centro Federal de Educação Tecnológica
CPU	-	*Central Processing Unit*
FO	-	Função Objetivo
IFCE	-	Instituto Federal do Ceará
JSON	-	*JavaScript Object Notation*
NP	-	*Nondeterministic Polynomial*
PDF	-	*Portable Document Format*
PAH	-	Problema de Alocação de Horários
PI	-	Programação Inteira
PIB	-	Programação Inteira Binária
PL	-	Programação Linear
PLI	-	Programação Linear Inteira
PLIM	-	Programação Linear Inteira Mista
PO	-	Pesquisa Operacional
PSO	-	*Particle Swarm Optimization*
RAM	-	*Random Access Memory*
SA	-	*Simulated Annealing*
TS	-	*Tabu Search*
UFF	-	Universidade Federal Fluminense

SUMÁRIO

1 Introdução **1**

2 Definições Gerais **3**

2.1 Programação Linear Inteira 3

2.2 Descrição do Problema 5

3 Trabalhos Relacionados **8**

4 Metodologia **16**

4.1 Formulação matemática do problema de alocação de horários 16

4.1.1 Definições de Conjuntos e Variáveis 16

4.1.2 Restrições Fortes 18

4.1.3 Restrições Fracas 20

4.1.4 Normalização *min-max* 22

4.1.5 Função objetivo 24

4.2 Ferramentas 25

4.2.1 *Solver IBM ILOG CPLEX* 25

4.2.2 Tecnologias para o *Back-end* 26

4.2.3 Tecnologias para o *Front-end* 27

4.3 Desenvolvimento da Interface do Usuário 29

5 Resultados **37**

6 Considerações Finais **40**

Apêndice A - Arquivo de entrada *JSON* **43**

Apêndice B - Grade horária gerada **44**

1 Introdução

A área de estudo abordada neste trabalho está inserida no campo da Pesquisa Operacional (PO), de forma mais específica no cenário de otimização combinatória aplicada ao problema de alocação de horários (PAH), também conhecido na literatura como *Timetabling Problem* (1). Segundo Hillier(2), como o próprio nome sugere, a Pesquisa Operacional envolve uma "pesquisa sobre operações". Assim, a PO é aplicada à problemas que dizem respeito a como conduzir e coordenar as operações dentro de uma organização.

De acordo com Andrade(3), pode-se considerar que o nome Pesquisa Operacional é de origem militar, tendo sido usada pela primeira vez durante a Segunda Guerra Mundial. Segundo ele, a PO é caracterizada, em termos científicos, por um campo de aplicações bastante amplo, o que justifica a existência de várias definições, algumas tão gerais que podem se aplicar a qualquer ciência, e outras tão particulares que só são válidas em determinadas áreas de aplicação, porém Andrade(3) também afirma que a PO é uma ciência aplicada que utiliza técnicas científicas conhecidas, ou as desenvolve quando necessário, tendo como ponto de referência a aplicação do método científico, desta forma, tem a ver com a pesquisa científica criativa em aspectos fundamentais referente às operações de uma organização.

Conforme Daskalaki, Birbas e Housos(4) o PAH é definido como o processo de atribuir disciplinas a períodos de tempo específicos ao longo dos cinco dias úteis da semana, de acordo com às necessidades específicas de cada disciplina e instituição.

Em concordância com El-Sherbiny et al.(5), o *timetabling problem* é um problema relevante do cotidiano, frequentemente encontrado em contextos educacionais, empresariais, em eventos esportivos e no setor de transportes. Segundo El-Sherbiny et al.(5), gerar uma solução para o PAH por métodos tradicionais, sendo estes geralmente processos manuais, exige um esforço significativo e um elevado tempo para satisfazer todas as restrições envolvidas. Ele surge como um problema de atribuição multidimensional, em que um professor é designado a lecionar uma disciplina em um local e horário específicos, respeitando um conjunto de restrições referentes à instituição.

O problema abordado se torna interessante pois, de forma não automatizada, os responsáveis pela criação da grade de horários gastam horas para encontrar uma solução satisfatória, e que na maioria das vezes não é uma solução ótima de acordo com os critérios considerados. De acordo com Vieira e Macedo(6), trata-se de um problema combinatório complexo, dado que o conjunto de interesses particulares da instituição, professores e alunos geram várias condições conflitantes a serem atendidas. Segundo Alghamdi et al.(7), de modo geral, resolver esses desafios e obter soluções ótimas exatas é computacionalmente intratável. Portanto, o PAH é um exemplo de problema NP-difícil.

Neste trabalho, o PAH será abordado por meio da formulação e resolução de um modelo matemático de Programação Linear Inteira (PLI). A PLI é uma técnica da PO orientada à resolução de problemas em que o objetivo é maximizar ou minimizar uma função linear sujeita a um conjunto de restrições também lineares, com a particularidade de que as variáveis de decisão devem assumir

valores inteiros. Essa metodologia permite representar as decisões envolvidas no problema, como a atribuição de disciplinas a horários, por meio da utilização de variáveis binárias e restrições que caracterizam as condições impostas pela instituição, como a disponibilidade de professores, conflitos entre disciplinas e carga horária semanal.

A resolução do modelo é realizada neste trabalho utilizando o *solver IBM ILOG CPLEX*, uma ferramenta reconhecida por sua eficiência na resolução de problemas de otimização de grande porte, capaz de encontrar soluções ótimas mesmo para problemas complexos. A escolha da PLI em conjunto com o *CPLEX* permite a resolução do problema de geração da grade de horários, assim como a implementação do controle preciso sobre as regras e particularidades do Cefet, facilitando modificações ou extensões do modelo.

Além da modelagem matemática e resolução do modelo, é proposto o desenvolvimento de uma interface de usuário utilizando as ferramentas *React* e *TypeScript*, em conjunto com um *back-end* em *Python*, responsável por traduzir os dados de entrada (como disciplinas, semestres, horários de aula, professores, dias letivos) para o formato esperado pelo modelo, executar o *solver* e retornar a grade de horários gerada. Dessa forma, a metodologia adotada proporciona uma integração entre modelagem matemática e tecnologias modernas de *software*, criando assim uma solução prática, flexível e reutilizável para o problema de alocação de grades de horário.

O principal objetivo deste trabalho é desenvolver um modelo matemático, baseado em PLI capaz de gerar automaticamente grades horárias acadêmicas viáveis para o curso de Engenharia de Computação do Centro Federal de Educação Tecnológica Celso Suckow da Fonseca *Campus* Petrópolis, respeitando as restrições da instituição e buscando otimizar critérios relevantes para a organização. O trabalho não se limita apenas à modelagem teórica, mas também desenvolve uma aplicação funcional que integra o modelo matemático a uma interface de usuário intuitiva. Desta maneira, o trabalho busca oferecer uma ferramenta que não apenas automatize o processo de geração de grades horárias, mas também o torne mais reproduzível e eficiente.

A importância da solução proposta está em reduzir o esforço manual e o tempo gasto que ocorrem em abordagens tradicionais, geralmente feitas por meio de planilhas. Ao utilizar a PLI em conjunto com as ferramentas citadas anteriormente, espera-se aumentar a precisão, consistência e qualidade das grades horárias geradas, visando garantir que conflitos de horários e violações de restrições sejam eliminados ou reduzidos significativamente. E para atingir esse objetivo, o trabalho se desenvolve em três núcleos principais:

- Formulação do modelo matemático com base nas regras e restrições inerentes ao curso de Engenharia de Computação do Cefet/RJ - *Campus* Petrópolis.
- Implementação computacional da solução utilizando o *solver IBM ILOG CPLEX* e a linguagem *Python*.
- Desenvolvimento de uma aplicação *front-end* com *TypeScript* e *React* que permita a interação com o sistema de forma simples.

2 Definições Gerais

Nesta seção, serão apresentadas as definições fundamentais utilizadas na modelagem matemática, assim como uma visão geral do problema abordado. O PAH é formulado como um modelo de PLI, sendo esta técnica muito utilizada para resolver problemas de alocação com múltiplas restrições. O objetivo principal consiste em encontrar uma distribuição viável e eficiente de disciplinas ao longo da semana. Para isso, são definidas variáveis binarias de alocação, bem como função objetivo (FO) e restrições fortes e fracas, que refletem os critérios definidos pela instituição.

2.1 Programação Linear Inteira

A Programação Linear Inteira é uma subárea da programação matemática que trata da resolução e problemas de otimização nos quais as variáveis de decisão devem assumir valores inteiros, diferentemente da programação linear (PL), na qual as variáveis podem assumir qualquer valor real dentro de um intervalo.

De acordo com Hillier(2), um modelo de PLI é formado por três elementos principais:

1. As variáveis de decisão com domínio inteiro.

2. A função objetivo, sendo representada por uma expressão matemática, em função das variáveis de decisão, na qual se deseja maximizar ou minimizar um determinado critério.

3. As restrições, sendo estas um conjunto de condições que a solução deve satisfazer. Elas podem ser restrições fortes (*hard constraints*), que não podem ser violadas, ou restrições fracas (*soft constraints*) que podem ser violadas com penalidade, permitindo flexibilidade na busca por uma solução de melhor qualidade.

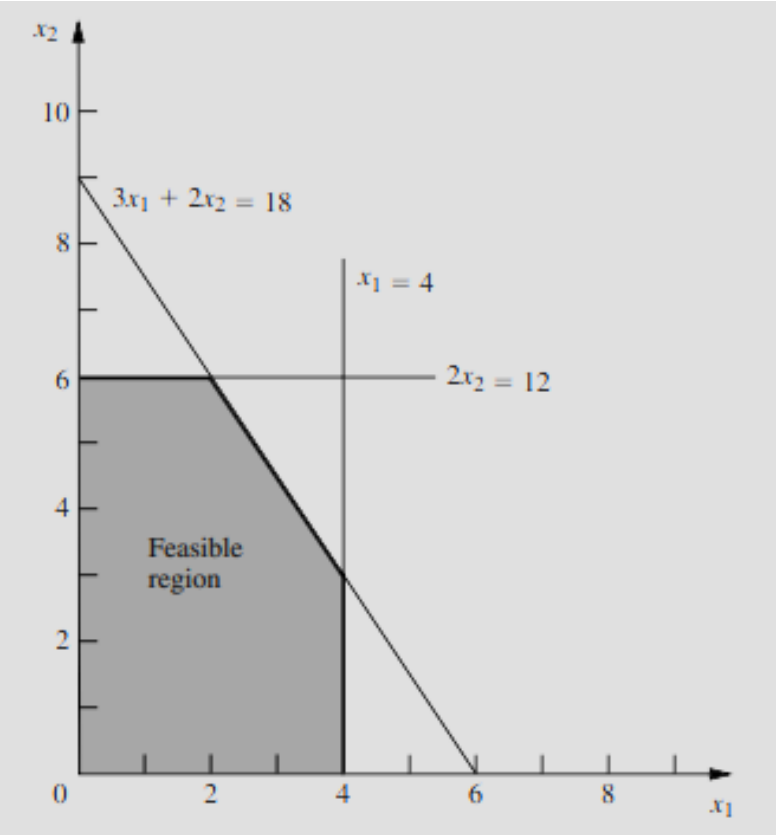
De forma geral, um exemplo de modelo de PLI pode ser descrito pela seguinte formulação:

$$\begin{aligned} \text{maximizar} \quad & Z = 3x_1 + 5x_2 \\ \text{sujeito a:} \quad & x_1 \leq 4 \\ & 2x_2 \leq 12 \\ & 3x_1 + 2x_2 \leq 18 \\ \text{e} \quad & x_1 \geq 0, \quad x_2 \geq 0 \end{aligned}$$

Onde a região viável, que consiste em todos os vetores x que satisfazem as restrições, pode ser observada na Figura 1.

Segundo Wolsey(8), técnicas de PLI demonstram ser uma forte ferramenta para modelar problemas combinatórios, nos quais existem um grande número de possibilidades possíveis, mas apenas algumas são válidas. No caso da alocação de horários acadêmicos, a PLI se mostra adequada por

Figura 1: Região viável de um modelo de PLI.



Fonte: Retirado de (2).

permitir a incorporação de regras operacionais ao mesmo tempo em que permite flexibilizar restrições secundárias por meio da minimização de penalidades associadas as mesmas.

Entre as vantagens da PLI estão a clareza, pois cada regra do sistema pode ser representada por uma fórmula matemática explícita, a flexibilidade devido a possibilidade de integrar diferentes tipos de restrições alinhados ao objetivo, sua adaptabilidade pois o modelo pode ser ajustado conforme mudanças nos dados ou nas prioridades específicas do Cefet, juntamente à sua capacidade de escalabilidade, ou seja, de encontrar soluções ótimas para instâncias de grande porte, pois *solvers* como o *CPLEX* e *Gurobi* são altamente eficientes, de acordo com Hillier(2) e (9), e podem resolver problemas reais com um grande número de variáveis e restrições.

Porém, apesar destas vantagens, a PLI também apresenta alguns desafios, estando entre eles o tempo de processamento pois problemas grandes podem ter um espaço de busca similarmente grande. Também faz-se necessário uma modelagem cuidadosa, visto que pequenas falhas na definição das variáveis ou restrições podem tornar o problema inviável ou sem sentido lógico, assim como a escalabilidade dado que embora os *solvers* sejam eficientes, há limites práticos para o tamanho das instâncias resolvíveis em tempo aceitável.

2.2 Descrição do Problema

O PAH é um desafio bastante recorrente em instituições educacionais, e de maneira geral ele consiste em alocar disciplinas, professores, horários e turmas de forma eficiente e organizada, certificando que todas as atividades acadêmicas sejam distribuídas de acordo com as exigências formais curriculares e operacionais da instituição. O PAH é considerado complexo e é classificado como um problema NP-difícil, devido a quantidade de combinações possíveis entre os elementos envolvidos e a necessidade de atender simultaneamente a múltiplas restrições.

O trabalho aqui desenvolvido realiza um estudo sobre este problema aplicado ao contexto do curso de Engenharia de Computação do Cefet/RJ - *Campus* Petrópolis, voltando-se para a construção da grade horária semanal de disciplinas obrigatórias e optativas dos diferentes, e ao total dez, períodos do curso.

A Figura 2 apresenta a matriz curricular do curso, na qual cada coluna representa um período letivo, e cada caixa representa uma disciplina ofertada no determinado período. Dentro de cada caixa estão o nome da disciplina, seu código e carga horária semanal de aulas teóricas e práticas, assim como seus pré-requisitos indicados pelos códigos localizados abaixo do nome da disciplina. As cores associadas a cada disciplina auxiliam na identificação de seus eixos temáticos, estando entre eles fundamentos em engenharia, fundamentos em computação, teleprocessamento e computação de alto desempenho, sistemas computacionais, prática científica e desenvolvimento humano, social e profissional. Há também o grupo de possíveis disciplinas optativas a serem ofertadas, localizadas na parte inferior da matriz. Dessa forma a matriz curricular oferece uma visão abrangente e estruturada do curso, proporcionando assim um maior entendimento para o problema abordado neste trabalho.

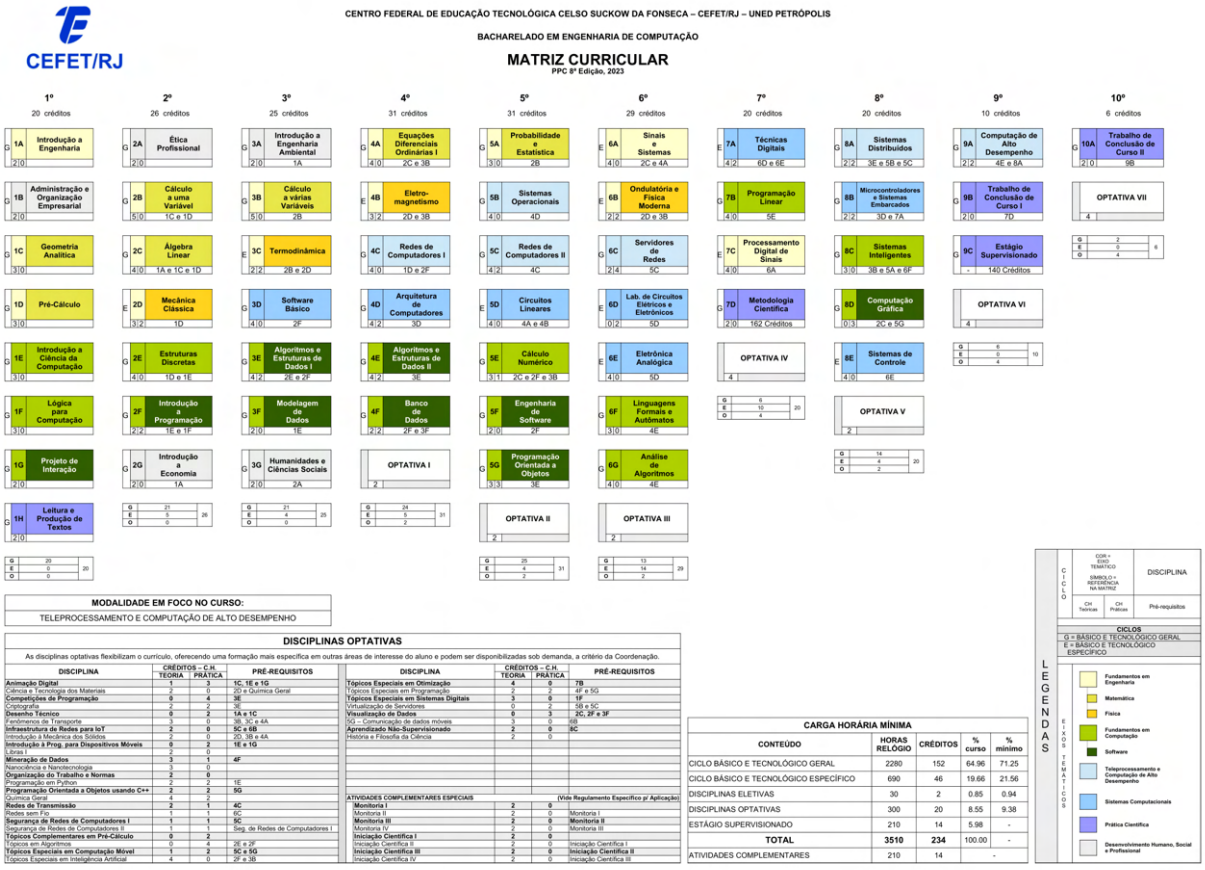
A aplicação prática deste trabalho ocorre na geração de uma grade horária automatizada, considerando: *(i)* disciplinas por semestre, *(ii)* carga horária obrigatória semanal, *(iii)* professores responsáveis por cada disciplina no período, *(iv)* dias e horários disponíveis para a realização das aulas, *(v)* preferências e restrições que influenciam diretamente na qualidade da grade horária gerada.

O processo de geração considera todos esses dados estruturados em um formato padronizado, que é interpretado pelo sistema para então resolver o problema por meio de técnicas de otimização.

O objetivo principal do sistema é produzir uma grade horária viável e eficiente que respeite regras curriculares e operacionais do Cefet. Além disso, busca-se obter uma solução equilibrada, que minimize choques de horário, respeite preferências e otimize a experiência tanto para os professores, quanto para os alunos do curso de Engenharia de Computação.

De acordo com Pereira et al.(1), uma solução para o PAH que viole sequer uma restrição forte se torna uma solução inviável, enquanto uma solução que viole qualquer número de restrições fracas ainda é considerada viável. As restrições fracas são utilizadas para melhorar a qualidade da solução, pois satisfazê-las significa que a solução atende a alguma(s) preferência(s) específica(s), mas elas não indicam regras obrigatórias. Portanto, é possível encontrar uma solução viável independentemente de atender ou não a uma restrição fraca. As restrições fortes são as que devem ser obrigatoriamente

Figura 2: Matriz Curricular - Engenharia de Computação.



atendidas pela solução fornecida.

As restrições fortes, isto é, que não podem ser violadas, consideradas para o problema deste trabalho em questão, são:

- A disciplina deve ser alocada de acordo com sua carga horária — a quantidade de blocos atribuídos a cada disciplina deve corresponder exatamente à sua carga semanal, sem excessos nem faltas.
- A disciplina só pode ser atribuída ao seu semestre específico — uma disciplina do 1º período, por exemplo, não pode ser alocada em horários do 3º período.
- No mesmo período, dia e horário, não pode haver mais de uma disciplina — duas disciplinas de um mesmo semestre não podem ocorrer simultaneamente.
- Um professor não pode ministrar duas disciplinas ao mesmo tempo — a grade horária gerada deve garantir que o professor esteja alocado em apenas um local por horário.
- Restrições de blocos por carga horária — disciplinas com cargas pares (por exemplo, 2, 4 ou 6 tempos) devem ser distribuídas em blocos distintos de 2 ou 4 tempos, enquanto disciplinas com

carga ímpar devem ocupar blocos de 2 ou 3 tempos.

Além das restrições fortes citadas anteriormente, o modelo também considera um conjunto de restrições fracas, que são critérios desejáveis, mas que podem ser violados por uma solução viável. Cada violação de uma restrição fraca adiciona penalidades à solução fornecida pelo modelo. Essas penalidades são minimizadas durante o processo de otimização. As restrições fracas são:

- Professores com aulas em mais de três dias diferentes da semana, buscando dessa maneira concentrar suas atividades e evitar sobrecarga.
- Uma disciplina e disciplinas que são pré-requisito desta primeira ocorrem em dias diferentes. O ideal é que essas disciplinas aconteçam no mesmo dia, dado que elas não podem ser cursadas simultaneamente.
- Disciplinas com alta taxa de reprovação que ocorrem no mesmo horário, analisadas no intervalo de períodos diretamente vizinhos à disciplina considerada. Essa restrição fraca tem por objetivo minimizar o choque de horário, dado que é provável que um aluno tente realizar ambas disciplinas no mesmo período.
- Alocação de aulas em dias que não são preferidos por cada professor, respeitando suas preferências individuais.
- Lacunas (*gaps*) na grade de um mesmo professor ao longo do dia, buscando uma distribuição mais compacta e eficiente de seu tempo.

Considerando todas as restrições citadas nesta seção, o modelo matemático tem por objetivo principal encontrar uma solução que satisfaça rigorosamente as restrições fortes, enquanto busca minimizar as penalidades associadas à violação das restrições fracas, resultando deste modo em uma grade horária equilibrada, funcional, eficiente e alinhada com os objetivos acadêmicos e operacionais do Cefet/RJ - *Campus* Petrópolis.

3 Trabalhos Relacionados

Na literatura, variações do problema de determinação ótima de grades de horários são amplamente estudados devido à sua relevância prática e complexidade combinatória que torna o problema desafiador em cenários práticos, por exemplo, em universidades com vários cursos e disciplinas que competem pelos professores e salas. Phillips et al.(10) propõem a resolução do PAH por meio da abordagem de pertubação mínima, e Hertz(11) que utiliza busca tabu para reduzir o número de conflitos entre disciplinas que ocorrem simultaneamente e envolvem estudantes ou professores em comum. Diferentes abordagens são exploradas, como métodos baseados em algoritmos exatos, como PI nos trabalhos de Phillips et al.(10) e Cataldo et al.(12), algumas abordagens heurísticas apresentadas por Ganguli e Roy(13) e Hambali, Olasupo e Dalhatu(14), como a coloração de grafos e algoritmos genéticos, e metaheurísticas como as apresentadas nos trabalhos de Daniel, Alimi e Modi(15) e Kahar e Kendall(16), estando entre elas o algoritmo do grande dilúvio.

Enquanto os algoritmos exatos garantem soluções ótimas para alguns problemas e instâncias, sua aplicação é limitada devido ao elevado custo computacional, principalmente ao lidar com instâncias de grande porte. Por outro lado, heurísticas e metaheurísticas, como algoritmos genéticos (17) e busca tabu (11), são amplamente utilizados por oferecerem soluções aproximadas de boa qualidade em tempos computacionais viáveis.

Foi realizada uma revisão da literatura sobre o assunto que envolve essa pesquisa e os artigos analisados foram selecionados por meio de buscas no Google Scholar[1], utilizando palavras-chave relacionadas ao problema de otimização de grades de horários, como:

- *Timetabling optimization*.
- *Integer linear programming for course scheduling*.
- Otimização de grades horárias.

Após a realização da primeira busca, que auxiliou na criação de um conjunto base inicial de trabalhos, foi utilizada a técnica de *snowballing* (18), que consiste em identificar novos trabalhos a partir da lista de referências e lista de citações dos artigos selecionados anteriormente para compor o conjunto base. Para tornar esse processo mais eficiente, foi utilizada a ferramenta *Research Rabbit*[2], pois facilita a descoberta de publicações relacionadas ao tema, a partir de um conjunto base de trabalhos, permitindo assim uma exploração mais ampla e direcionada da literatura existente. Essa abordagem garantiu a seleção de artigos que cobrem métodos e aplicações dentro do escopo desta pesquisa.

Os artigos examinados apresentam diferentes metodologias para a resolução do PAH, destacando tanto o uso de modelos matemáticos baseados em programação linear inteira (PLI) quanto a

[1] https://scholar.google.com.br/
[2] https://researchrabbitapp.com/

aplicação de heurísticas e metaheurísticas. Cada um desses trabalhos contribui para a compreensão do problema, das técnicas disponíveis e das dificuldades associadas à otimização de cronogramas universitários, oferecendo percepções importantes para a formulação do modelo proposto neste trabalho.

Ganguli e Roy(13) propuseram o uso de algoritmos de coloração de grafos como abordagem heurística para resolver o problema de grade horária. Um grafo é construído onde os vértices representam disciplinas e a coloração adequada desse grafo gera uma alocação de horários sem conflitos. Além disso, o artigo discute a inclusão de restrições como a indisponibilidade de professores, preferências de horários e agrupamento de aulas consecutivas para professores que oferecem duas aulas da mesma disciplina. Um exemplo prático é apresentado com disciplinas obrigatórias e optativas, demonstrando como a coloração do grafo pode gerar soluções ótimas. O artigo conclui que embora existam diversas soluções ótimas, a melhor delas é aquela que maximiza a satisfação das restrições fracas, e que a complexidade do problema cresce com o número de restrições impostas, e encontrar algoritmos que gerem soluções únicas e otimizadas continua sendo um desafio relevante e promissor para pesquisas futuras.

No artigo de Hambali, Olasupo e Dalhatu(14) é apresentado uma abordagem heurística para o problema de agendamento de horários de aulas na Universidade Federal de Wukari, utilizando uma combinação de algoritmos genéticos e *simulated annealing* (SA). As restrições fracas incluem algumas regras específicas da instituição como o encerramento das atividades às quartas às 15 horas e às sextas entre 13 e 15 horas. A solução utiliza a teoria da evolução por seleção natural para gerar e evoluir horários viáveis por meio de múltiplas iterações. O *software* implementado se demonstrou eficiente produzindo horários livres de conflitos em apenas oito iterações, com base nos dados da própria faculdade. Além de aplicável a outras instituições com características similares, o sistema é flexível permitindo personalização conforme restrições específicas.

No trabalho apresentado por Hertz(11), o autor propõe uma abordagem baseada na técnica de busca tabu para resolver problemas de agendamento de horários em larga escala. O objetivo está na redução de conflitos entre cursos realizados simultaneamente por alunos, além da compactação de horários devido a preferência dos professores por estarem alocados em períodos consecutivos de tempo. A metodologia é dividida em duas fases, que consiste em encontrar uma solução viável que satisfaça todas as restrições fortes e logo após aprimorar a solução atendendo o máximo de restrições fracas possíveis. A abordagem demonstrou um bom desempenho quando aplicada em dados reais, como o agendamento de 288 cursos da Faculdade de Economia da Universidade de Genebra, com 143 professores, 1.729 alunos e 67 salas distribuídas em 15 prédios.

Phillips et al.(10) apresentam um modelo de perturbação mínima para resolver inviabilidades sem comprometer a grade horária já existente, demonstrando a eficácia da PLI para ajustes localizados, por meio da realização de testes com dados reais da Universidade de Auckland. Nesse contexto, o modelo busca não apenas atribuir horários e salas para todos os eventos do curso de forma viável, como também minimizar o impacto de alterações necessárias que surgem ao longo das fases de construção e implementação do cronograma. O modelo considera duas etapas, sendo a primeira *(i)* maximizar

o número total de eventos atribuídos a horários e salas de forma viável, e a segunda etapa busca *(ii)* minimizar a perturbação em relação ao cronograma original, por meio de coeficientes de penalidade que quantificam o impacto de mover eventos entre horários ou salas.

De forma semelhante Pereira et al.(1) utilizam PLI para priorizar a alocação de professores mais titulados, de forma que professores com mais títulos acadêmicos possuem uma chance maior de serem alocados em disciplinas. A PLI também é utilizada para otimizar a distribuição de disciplinas, considerando restrições fracas como *(i)* a preferência de agrupar turmas de diferentes cursos com ementas equivalentes, *(ii)* agrupar módulos ministrados por um mesmo professor no mesmo dia da semana, e *(iii)* a preferência de docentes que ministram apenas um módulo no dia estejam alocados no horário da manhã. A formulação matemática alcançou uma solução em 4 minutos e 21 segundos e o modelo foi bem aceito por gestores e coordenadores de curso de uma faculdade do Rio de Janeiro, evidenciando desta forma a aplicabilidade prática do modelo proposto em instituições de ensino com condições semelhantes.

Além das abordagens baseadas em programação linear (PL), Alghamdi et al.(7) apresentam uma revisão da literatura com as principais técnicas utilizadas no problema de grade horária acadêmica como *Particle Swarm Optimization* (PSO), busca tabu, SA, PI, algoritmo do grande dilúvio, entre outros, destacando a classificação entre algoritmos exatos, heurísticos e metaheurísticos e reforçando a complexidade NP-difícil do problema, dado que é um problema de otimização combinatória com um vasto espaço de busca e geralmente um grande número de restrições são consideradas.

Adicionalmente, Queiroz et al.(19) desenvolveu um modelo matemático voltado para atender às preferências dos professores do curso de Engenharia de Produção Civil do Instituto Federal do Ceará (IFCE) *campus* Quixadá. Queiroz et al.(19) abordam a maximização das preferências dos professores em relação à sequência e quantidade de dias de ensino, garantindo que, mesmo sem atender integralmente às preferências sequenciais, todos os docentes tenham sua carga horária distribuída de acordo com os grupos de dias escolhidos. A modelagem foi implementada utilizando o *solver CPLEX* e os dados foram extraídos de planilhas fornecidas pela coordenação do curso. Durante a simulação o tempo de execução foi estabilizado em 2 minutos, critério adotado como tempo máximo de parada para obtenção de uma solução viável, e os resultados mostraram que mesmo quando não foi possível atender à exata preferência de dias sequenciais de todos os docentes, o modelo garantiu que todos fossem alocados dentro do grupo preferido de dias.

Já Bucco, Bornia-Poulsen e Bandeira(20) propõem um modelo matemático baseado em Programação Linear Inteira Mista (PLIM) para otimizar a utilização do espaço físico, reduzindo a quantidade de espaços necessários e minimizar custos operacionais, como o aluguel de salas de aulas, que chegavam a 1.6 milhões de dólares. Os dados de teste foram coletados de uma universidade federal brasileira e o modelo foi dividido em duas fases principais: *(i)* a construção do horário semanal, sem considerar inicialmente a alocação de salas, e posteriormente *(ii)* a atribuição das salas de aula buscando minimizar o uso de unidades acadêmicas com objetivo de reduzir custos. Para resolver o modelo foi desenvolvido um algoritmo implementado em C++ e resolvido com o *CPLEX*. Os resul-

tados mostraram que ao distribuir melhor as aulas nos horários disponíveis a demanda por salas foi significativamente reduzida, o que permitiu à instituição evitar o aluguel de novos prédios.

Daskalaki, Birbas e Housos(4) propõem uma modelo de Programação Inteira Binária (PIB) para o problema de elaboração de grades de horários. O objetivo principal é minimizar uma função linear de custos, permitindo incorporar restrições relacionadas a períodos e dias da semana ao mesmo tempo em que reduz o espaço de busca. As restrições fracas que caracterizam o problema se dão por meio da preferência de horários por parte dos docentes, a compactação das grades dos alunos e a redução na mudança de salas. Como estudo de caso, a formulação foi aplicada no Departamento de Engenharia Elétrica e de Computação da Universidade de Patras, na Grécia. Três testes com diferentes tamanhos de instâncias foram realizados, com número de cursos variando entre 25 e 92 e obtendo soluções em tempos que variam de 2 minutos e meio a 95 minutos. Os autores concluem que apesar da complexidade natural do problema, a abordagem é viável para problemas reais e de grande porte, fornecendo soluções ótimas e de alta qualidade, com o modelo mostrando que a automação da construção de horários é possível e vantajosa, substituindo um processo historicamente manual e trabalhoso por uma solução computacional eficiente.

Além dos estudos mencionados, outras abordagens matemáticas também foram exploradas para otimizar a alocação de horários acadêmicos. No caso da Universidade de Hannover, o modelo proposto por Schimmelpfeng et al.(21) utilizou PI para atribuir horários e salas às disciplinas, levando em consideração um grande número de restrições institucionais, tais como: *(i)* número máximo de dias para professores, *(ii)* intervalos entre dias de aula, *(iii)* preferências por horários de aula, dentre outras. Os resultados obtidos com a implementação do modelo demonstraram que um planejamento baseado em otimização gera cronogramas melhores do que aqueles anteriormente utilizados, garantindo um equilíbrio entre as necessidades institucionais e as preferências dos professores.

Por outro lado, Cataldo et al.(12) tratam especificamente da criação de horários de provas, abordando a complexidade do problema ao considerar vários cursos simultaneamente. A solução proposta combina PLI com técnicas de clusterização, reduzindo a complexidade computacional e garantindo uma melhor distribuição das avaliações. O modelo conseguiu minimizar conflitos e otimizar o uso dos espaços disponíveis, garantindo que os exames ocorressem em salas próximas, tornando o processo mais fácil para os alunos e eficiente para os gestores acadêmicos.

Dando continuidade às abordagens matemáticas para alocação de horários, o trabalho de Gall e Silva(22) propõe um modelo para a construção de grades horárias para o curso de Ciência da Computação da Universidade Federal Fluminense (UFF), *campus* Rio das Ostras. A formulação matemática utilizada busca otimizar a distribuição de turmas e professores, considerando as restrições impostas pela instituição como a minimização do excesso de aulas em um dia e as preferências dos docentes. Foram realizados cinco testes, sendo eles com base no *(i)* número de salas, *(ii)* dias evitados da semana, *(iii)* proibição de horários específicos, *(iv)* remoção do máximo de aulas por dia e *(v)* flexibilização do modelo, onde as mesmas amostras de entrada foram utilizadas para possibilitar a comparação entre as soluções do modelo com e sem a flexibilização dos perfis de turma. Os resultados demonstraram

que a flexibilização do modelo melhorou a qualidade das soluções geradas, tornando o processo mais ágil e consistente. O modelo também reduziu significativamente o tempo necessário para a tarefa de geração da grade de horários.

No trabalho de MirHassani(23), a PI é explorada para a resolução do PAH no contexto da faculdade de ciências da Universidade de Shahrood. O modelo binário proposto permitiu a incorporação de várias regras operacionais, como a de que disciplinas com mais de uma sessão semanal não podem ocorrer em dias consecutivos, além de algumas restrições fracas como a preferência de turnos e dias da semana por parte dos docentes, garantindo uma atribuição adequada dos horários aos professores, disciplinas e salas. O modelo foi implementado no *software* de modelagem e otimização AIMMS[3], utilizando o *solver XA*[4]. Os resultados indicam que o modelo proposto supera as metodologias tradicionais, como a utilizada anteriormente na universidade, além de apresentar resultados satisfatórios frente a métodos já estabelecidos na literatura, proporcionando soluções mais robustas e flexíveis.

Seguindo essa linha de pesquisa, Akkoyunlu(24) apresenta um modelo matemático de PL para a criação de grades horárias dentro de um único departamento universitário. O modelo associa custos a diferentes características do cronograma, como preferência de turnos e tamanho do intervalo entre aulas por parte dos docentes, e conflitos relacionados às disciplinas que possuem alta probabilidade dos estudantes desejarem cursá-las simultaneamente, buscando assim, minimizar o impacto dessas atribuições. Porém, a definição desses custos apresenta desafios, pois pequenas variações podem resultar em soluções muito diferentes. Para evitar cronogramas injustos, onde um grupo específico de alunos ou professores é penalizado, o estudo propõe a geração de múltiplas soluções quase ótimas, permitindo maior flexibilidade na escolha do melhor horário.

A Tabela 1 apresenta uma comparação entre algumas das diferentes abordagens propostas na literatura para a resolução do PAH. Na primeira coluna são indicadas as referências dos trabalhos analisados. A segunda coluna descreve a função objetivo utilizada em cada abordagem, enquanto a terceira coluna lista algumas das principais restrições consideradas nos modelos. A quarta coluna apresenta o método utilizado para a resolução do problema, e, por fim, a quinta coluna informa quais dados foram utilizados para a realização dos testes.

Tabela 1: Comparação entre diferentes abordagens para o problema de grade de horários.

Ref.	**FO**	**Restrições**	**Método**	**Dados**
(4)	Minimizar penalidades de restrições fracas	Preferências dos docentes e grade compacta	PIB	Dados reais da Universidade de Patras

Continua na próxima página

[3]https://www.aimms.com/

[4]https://www.maximalsoftware.com/solvers/xa.html

Ref.	FO	Restrições	Método	Testes
(1)	Maximizar alocação de professores em função de peso atribuído a cada um	Preferência de alocação de professores com mais títulos acadêmicos	PLI	Dados reais de uma faculdade no Rio de Janeiro
(10)	Minimizar a perturbação do cronograma proposto e o cronograma inicial	Impor contiguidade temporal em eventos longos	PLI	Dados reais da Universidade de Auckland
(11)	Minimizar número de conflitos entre cursos realizados simultaneamente por alunos	Compacidade da grade horária	*Tabu search*	Dados reais da Universidade de Genebra
(12)	Minimizar penalidades de restrições fracas	Provas de disciplinas de períodos consecutivos de um mesmo curso não devem ser agendados no mesmo horário	PLI com técnicas de *Clustering*	Dados reais da Universidade Diego Portales
(13)	Maximizar a satisfação das restrições fracas	Não mais do que duas aulas teóricas consecutivas podem ser atribuídas ao mesmo professor para ensinar a mesma disciplina.	Algoritmo de coloração de grafos	Dados reais de uma universidade da Índia
(14)	Minimizar penalidades de restrições fracas	Encerramento das atividades às quartas às 15 horas e às sextas entre 13 e 15 horas.	Algoritmos Genéticos e SA	Dados reais da *Federal University Wukari*
(15)	Minimizar conflitos de horário	As disciplinas gerais da faculdade devem ser alocadas antes das disciplinas específicas de cada departamento	Algoritmo Quicksort	Dados reais da *Kebbi State University of Science and Technology*

Continua na próxima página

Ref.	FO	Restrições	Método	Testes
(19)	Maximizar preferências dos professores em relação à sequência e quantidade de dias de ensino	Preferências dos docentes	PLI	Dados reais do curso de Engenharia de Produção Civil do Instituto Federal do Ceará (IFCE) *campus* Quixadá
(20)	Minimizar o número de aulas atribuídas ao intervalo de tempo com a maior quantidade de reuniões	Redução de custos de utilização das unidades acadêmicas	PLIM	Dados reais de uma universidade federal brasileira
(21)	Minimizar penalidades de restrições fracas	Número máximo de dias para professores, intervalos entre dias de aula	PI	Dados reais da Universidade de Hannover
(22)	Minimizar penalidades de restrições fracas	Redução do excesso de aulas em um dia e preferências dos docentes	PI	Dados reais do curso de Ciência da Computação da Universidade Federal Fluminense, *campus* Rio das Ostras
(23)	Minimizar penalidades de restrições fracas	Disciplinas com mais de uma sessão semanal não podem ocorrer em dias consecutivos	PLI	Dados reais da *Shahrood University of Technology*
(24)	Minimizar penalidades de restrições fracas	Preferência de tamanho do intervalo entre aulas por parte dos docentes e conflitos entre disciplinas com alta probabilidade de serem realizadas simultaneamente	PLI	Dados genéricos

Com base nos artigos analisados, se observa que a alocação de horários em instituições de ensino superior (com suas diversas variações) é um problema amplamente estudado por diferentes pesquisadores, os quais utilizam diferentes metodologias para resolver tal problema. Enquanto alguns trabalhos priorizam a otimização da distribuição de professores e turmas, outros focam na alocação eficiente de salas e na minimização de conflitos nas grades horárias. A utilização de técnicas como

PL, PLI, PLIM, *tabu search*, algoritmo de coloração de grafos, algoritmos genéticos, dentre outras, demonstra a complexidade do problema e a necessidade de soluções flexíveis para os diferentes contextos institucionais. Além disso, a adaptação dos modelos às particularidades de cada instituição, seja considerando as preferências dos docentes, a otimização do uso do espaço físico ou a minimização de sobrecarga de horários, reforça a importância de uma abordagem mais específica e personalizada. Assim, os trabalhos examinados fornecem uma base sólida para o entendimento do PAH, e desenvolvimento de novos modelos capazes de atender melhor às demandas específicas de instituições de ensino distintas, proporcionando a geração de uma grade de horários mais equilibrada e eficiente.

4 Metodologia

Neste capítulo será apresentada a metodologia proposta para solução do PAH utilizando dados obtidos a partir do curso de Engenharia de Computação do Cefet/RJ. Inicialmente, será apresentado o modelo matemático de PLI do problema em estudo, detalhando as variáveis de decisão, restrições e função objetivo. Logo após, serão descritas as ferramentas e tecnologias aplicadas para a implementação da solução, como o *CPLEX*[5] para resolução do modelo PLI, a linguagem *Python*[6] para o desenvolvimento do *back-end* e manipulação dos dados, juntamente com *TypeScript*[7] e *React*[8] para o desenvolvimento da interface de usuário interativa.

4.1 Formulação matemática do problema de alocação de horários

Esta seção apresenta o modelo matemático proposto para a resolução do PAH. São definidas as variáveis de decisão, a FO e os conjuntos de restrições fortes e restrições fracas, estruturadas de acordo com as especificações da instituição.

O modelo matemático proposto e desenvolvido neste trabalho tem por objetivo principal resolver o problema de determinação da solução ótima da grade horária semanal de um curso de graduação, porém usando como estudo de caso os dados do curso de Engenharia de Computação do Cefet/RJ - *Campus* Petrópolis, garantindo uma distribuição coerente e otimizada das aulas ao longo dos dias e horários disponíveis. Para isso, o modelo é formulado como um problema de PLI, uma abordagem amplamente utilizada em problemas de alocação e otimização combinatória, dada sua capacidade de representar restrições complexas e objetivo de forma estruturada.

4.1.1 Definições de Conjuntos e Variáveis

Para o modelo PLI proposto nesta pesquisa definimos os seguintes conjuntos e variáveis:

- P: conjunto de períodos do curso, $P = \{p_1, p_2, \cdots, p_{10}\}$.
- D: conjunto de dias letivos em uma semana, $D = \{d_1, d_2, \cdots, d_5\}$.
- H: conjunto de *slots* de horários disponíveis para uma aula, $H = \{h_1, h_2, \cdots, h_{12}\}$.
- H_{aft}: conjunto de *slots* de horários disponíveis para uma aula na parte da tarde, $H_{aft} = \{h_1, h_2, \cdots, h_6\}$.
- M: conjunto de disciplinas do curso, $M = \{m_1, m_2, \cdots, m_n\}$.
- M_{obrig}: conjunto de disciplinas obrigatórias do curso, $M_{obrig} = \{m_1, m_2, \cdots, m_{55}\}$.

[5] https://www.ibm.com/br-pt/products/ilog-cplex-optimization-studio
[6] https://www.python.org/
[7] https://www.typescriptlang.org/
[8] https://react.dev/

- M_{txr}: conjunto de disciplinas com alta taxa de reprovação, $M_{txr} = \{m_1, m_2, \cdots, m_t\}$.

- C: conjunto de cargas horárias, $C = \{c_1, c_2, \cdots, c_n\}$.

- F: conjunto de professores do curso, $F = \{f_1, f_2, \cdots, f_g\}$.

- K: conjunto de tamanhos de blocos de alocação permitidos para as disciplinas de acordo com sua carga horária semanal, $K = \{k_1, k_2, k_3\}$.

- B_k^s: variável binária que indica se uma disciplina está alocada em um bloco de tamanho k iniciando no bloco de *slots* s.

- P_{req}: valor da penalidade quando uma disciplina m_i e suas disciplinas que são pré-requisito desta ocorrem em dias diferentes.

- P_{fail}: valor da penalidade aplicada quando disciplinas com alta taxa de reprovação ocorrem no mesmo *slot* de tempo.

- P_{days}: valor da penalidade aplicada quando professores possuem aulas alocadas em mais de três dias da semana.

- P_{gaps}: valor da penalidade aplicada quando ocorrem lacunas na grade de um mesmo professor ao longo do dia.

- P_{pref}: valor da penalidade aplicada quando professores possuem aulas alocadas em dias que não são de sua preferência.

- $P_{afternoon}$: valor da penalidade aplicada quando uma matéria obrigatória é alocada na parte da tarde.

- Y_{fd}: para cada professor f e dia d, define-se esta variável binária, onde:

$$Y_{fd} = \begin{cases} 1, & \text{se o professor } f \text{ leciona no dia } d, \\ 0, & \text{caso contrário.} \end{cases} \tag{1}$$

- L_{fd}: para cada professor f e dia d, define-se esta variável binária, onde:

$$L_{fd} = \begin{cases} 1, & \text{se o professor } f \text{ leciona em um dia } d \text{ não preferido,} \\ 0, & \text{caso contrário.} \end{cases} \tag{2}$$

- A_{pdhm}: para cada período p, dia d, horário da tarde $h \in H_{aft}$ e disciplina obrigatória m, define-se esta variável binária, onde:

$$A_{pdhm} = \begin{cases} 1, & \text{se a disciplina obrigatória } m \text{ for alocada no dia } d, \text{ horário } h, \\ 0, & \text{caso contrário} \end{cases} \tag{3}$$

O modelo busca uma solução ótima, minimizando penalidades associadas a restrições fracas, ao mesmo tempo em que satisfaz de maneira obrigatória todas as restrições fortes do problema. O conjunto de variáveis de decisão do modelo é definido como uma matriz de 4 dimensões, conforme mostrado na Equação (4).

$$T_{pdhm} = \begin{cases} 1, & \text{se a disciplina } m \text{ está alocada no período } p, \text{ dia } d \text{ e horário } h, \\ 0, & \text{caso contrário.} \end{cases} \tag{4}$$

Esse conjunto de variáveis binárias T_{pdhm} representa a alocação de uma disciplina m, do período p, no dia d, e no horário h. Logo, para o nosso exemplo em específico se o modelo atribuir $T_{1111} = 1$ isso indica que a disciplina Introdução a Engenharia do primeiro período foi alocada na segunda-feira no horário de 7h às 7h50.

4.1.2 Restrições Fortes

O modelo possui o seguinte conjunto de restrições fortes:

- Toda disciplina deve ser alocada de acordo com sua carga horária:

$$\sum_{d \in D} \sum_{h \in H} T_{p(m)dhm} = C_m \quad \forall m \in M \tag{5}$$

A Restrição (5) assegura que cada disciplina $m \in M$ deve ser alocada exatamente C_m vezes, que representa sua carga horária semanal, respeitando o período ao qual pertence. O primeiro somatório percorre os dias letivos da semana e o segundo percorre os horários disponíveis em cada dia. Dessa forma, o lado esquerdo da equação representa a quantidade total de alocações da disciplina m, e a igualdade garante que essa quantidade seja compatível com a sua carga horária.

- A disciplina só pode ser alocada em seu respectivo período:

$$\sum_{\substack{p \in P \\ p \neq p(m)}} \sum_{d \in D} \sum_{h \in H} T_{pdhm} = 0 \quad \forall m \in M \tag{6}$$

A Restrição (6) garante que cada disciplina m seja alocada exclusivamente no período que pertence. O somatório externo percorre todos os períodos $p \in P$ diferentes de $p(m)$, enquanto o segundo somatório percorre os dias letivos e o terceiro os horários disponíveis para cada dia. Assim, o lado esquerdo da equação representa todas as possíveis alocações da disciplina m em períodos incorretos. Ao exigir que essa soma seja igual a zero, garante-se que a disciplina não seja atribuída a nenhum período que não seja o seu.

- As disciplinas de um mesmo período, não podem ser alocadas ao mesmo dia e horário:

$$\sum_{m \in M_p} T_{pdhm} \leq 1 \quad \forall p \in P,\ \forall d \in D,\ \forall h \in H \tag{7}$$

A Restrição (7) garante que, para cada combinação de período p, dia d e horário h, no máximo uma disciplina seja alocada. O somatório percorre o conjunto M_p, que representa todas as disciplinas pertencentes ao período p. Dessa forma, é evitado a sobreposição de aulas de disciplinas do mesmo período.

- Um professor não pode lecionar duas disciplinas ao mesmo tempo:

$$\sum_{m \in M_{\mathrm{f}}} T_{p(m)dhm} \leq 1 \quad \forall\, f \in F,\ \forall d \in D,\ \forall h \in H \tag{8}$$

A Restrição (8) certifica que cada professor f só esteja alocado a uma disciplina por vez. O conjunto M_f representa todas as disciplinas atribuídas ao professor f. Para cada combinação de professor, dia e horário, o somatório verifica todas as suas disciplinas, garantindo que ele esteja alocado em no máximo uma delas naquele instante.

- Blocos de alocação permitidos por dia:

A alocação de horários de uma disciplina m em um mesmo dia d deve formar blocos de tamanhos permitidos, conforme sua carga horária:

  - Se a disciplina possui uma quantidade par de tempos (carga horária), esta é alocada em 2 ou 4 *slots* de tempo.
  - Se a disciplina possui uma quantidade ímpar de tempos, esta é alocada em 2 ou 3 *slots* de tempo.

$$\sum_{h \in H} T_{pdhm} = \sum_{k(m) \in K} k \cdot \sum_{s \in S_k} B^s_{k(m)} \quad \forall m \in M,\ \forall d \in D \tag{9}$$

Na Equação (9), o lado esquerdo soma todos os tempos alocados à disciplina m no dia d, enquanto o lado direito representa a soma dos blocos possíveis, onde k é o tamanho do bloco e B^s_k indica se o bloco s de tamanho k foi utilizado.

– No máximo um bloco de disciplina por dia:

$$\sum_{k(m)\in K}\sum_{s\in S_k} B^s_{k(m)} \leq 1 \quad \forall m \in M,\ \forall d \in D \tag{10}$$

A Equação (10) assegura que não ocorra a fragmentação de horas ao longo do dia, garantindo blocos únicos e contínuos.

– Nenhuma disciplina pode ultrapassar 4 horários no mesmo dia:

$$\sum_{h\in H} T_{p(m)dhm} \leq 4 \quad \forall m \in M,\ \forall d \in D \tag{11}$$

Por fim, a Equação (11) define o limite superior de alocação diária por disciplina, evitando a sobrecarga em um único dia.

4.1.3 Restrições Fracas

No modelo são consideradas as seguintes restrições fracas, que são desejáveis, porém não inviabilizam a solução. Entretanto a violação destas restrições são penalizadas na FO, com o intuito de melhorar a qualidade final da grade horária gerada:

- Os professores não devem ser alocados em mais de 3 dias na semana:

  Podemos calcular a quantidade de dias que um professor leciona através da Equação (12).

$$\mathrm{dt}_f = \sum_{d\in D} Y_{fd} \tag{12}$$

  A violação por exceder três dias é modelada como:

$$\mathrm{excess_days}_f \geq \mathrm{dt}_f - 3 \tag{13}$$

  A desigualdade é utilizada para garantir que a variável nunca assuma valores negativos, já que ela é declarada com limite inferior igual a 0 (zero). Assim, caso $dt_f < 3$ a desigualdade garante que $excess_days_f = 0$, certificando que nenhuma penalidade seja associada a esses casos.

  A penalidade associada a essa violação é proporcional ao valor de *excess_days*, e será incorporada à FO da seguinte forma:

$$pen_excess = \sum_{f\in F} \mathrm{excess_days}_f \tag{14}$$

- Não deve ocorrer intervalos vazios entre aulas de um mesmo professor no mesmo dia:

Define-se uma variável binária gap_{fdi} para representar a existência de uma janela entre dois horários consecutivos:

$$\text{gap}_{fdi} \geq \sum_{m \in M_f} T_{p(m)dh_i m} - \sum_{m \in M_f} T_{p(m)dh_{i+1} m} \tag{15}$$

Onde h_i e h_{i+1} são horários consecutivos, M_f é o conjunto de disciplinas do professor f e $p(m)$ o período da disciplina m, garantindo assim que, se o professor leciona em h_i mas não leciona em h_{i+1}, então gap_{fdi} contará como uma violação, indicando a presença de uma janela entre esses horários.

- Professores não devem ser alocados em dias não preferidos:

  Para cada professor f e dia d que não esteja entre suas preferências, define-se:

$$L_{fd} \geq Y_{fd}, \quad \forall d \notin \text{pref}_f \tag{16}$$

  Ou seja, se o professor f leciona no dia d, e esse dia não está entre suas preferências, então a variável L_{fd} indicará uma violação da preferência.

  A penalidade total associada a alocações em dias não preferidos é dada por:

$$\text{pen_pref} = \sum_{f \in F} \sum_{d \in D \setminus \text{pref}_f} L_{fd} \tag{17}$$

- Disciplinas obrigatórias não devem ser alocadas no turno da tarde:

  Para cada disciplina obrigatória m, define-se a penalidade:

$$A_{pdhm} \geq T_{pdhm}, \quad \forall m \in M_{\text{obrig}}, \forall h \in H_{\text{aft}}, \forall\, d \in D \tag{18}$$

  Essa restrição assegura que, se uma disciplina obrigatória for alocada em um horário da tarde, a variável A_{pdhm} caracterizará uma violação da preferência por evitar alocações no turno da tarde.

  A penalidade associada a essas alocações é então computada como:

$$pen_aft = \sum_{p \in P} \sum_{d \in D} \sum_{h \in H_{\text{aft}}} \sum_{m \in M_{\text{obrig}}} A_{pdhm} \tag{19}$$

- Pré-requisitos de uma disciplina não devem ser alocados em dias diferentes:

  Para cada par de disciplinas (m_a, m_b), em que m_a é pré-requisito de m_b, define-se uma penalidade quando ambas não são alocadas no mesmo dia d.

A Equação (20) é utilizada para identificar essa violação.

$$\text{pen_prereq}_{m_a m_b d} = (\min(4, C_{m_a}) + \min(4, C_{m_b})) - \sum_{h \in H} T_{p_{m_a} dhm_a} - \sum_{h \in H} T_{p_{m_b} dhm_b} \tag{20}$$

- Disciplinas com alta taxa de reprovação não devem ser alocadas no mesmo dia e horário:

  A alocação simultânea de disciplinas com alta taxa de reprovação pode causar conflitos de horários, impedindo que alunos cursem essas matérias no mesmo período. Para evitar esse cenário, define-se uma penalidade sempre que as duas disciplinas forem alocadas no mesmo horário h e no mesmo dia d. Além disso, a penalidade é aplicada apenas quando essas disciplinas pertencem a períodos consecutivos, ou seja, $|p_{m_1} - p_{m_2}| = 1$.

  A violação é calculada pela Equação (21).

$$\text{pen_fail} = \sum_{\substack{m_1, m_2 \in M_{\text{txr}} \\ m_1 \neq m_2,\ |p_{m_1} - p_{m_2}| = 1}} \sum_{d \in D} \sum_{h \in H} T_{p_{m_1} dhm_1} \cdot T_{p_{m_2} dhm_2} \tag{21}$$

4.1.4 Normalização *min-max*

Com o objetivo de manter as penalidades dentro de uma escala comparável e equilibrada na FO, a normalização *min-max* é utilizada. Esta normalização transforma um valor bruto x em um valor no intervalo $[0, 1]$, usando a fórmula (22).

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{22}$$

Onde, x é o valor original da variável que será normalizada, x_{min} é o menor valor possível de x, x_{max} é o maior valor possível de x e x_{norm} é o valor normalizado obtido.

Porém, no modelo desenvolvido neste trabalho os valores mínimos são sempre 0 (zero), ou seja, quando não ocorre nenhuma violação das restrições fracas, logo a Equação 22 pode ser reduzida conforme apresentado em (23).

$$x_{\text{norm}} = \frac{x}{x_{\max}} \tag{23}$$

Essa abordagem garante que as violações das restrições estarão no mesmo intervalo de valores, e todas as penalidades relacionadas a cada restrição fraca sempre terá um peso proporcional ao seu impacto.

O cálculo dos valores máximos relacionados a cada penalidade são definidos a seguir:

- Máximo de penalidade por professores que lecionam em mais de 3 dias:

$$\text{max_excess_days} = \max\left(1, \sum_{f \in F} \max(0, |D| - 3)\right) \tag{24}$$

Para cada professor f, o número máximo de dias além do limite de 3 é $|D| - 3$. Soma-se isso para todos os professores, considerando o caso extremo em que podem ser alocados todos os dias.

- Máximo de penalidade por *gaps* entre aulas de um mesmo professor:

$$\text{max_gaps} = \max\left(1, (|H| - 1) \cdot |D| \cdot |F|\right) \tag{25}$$

Um *gap* pode ocorrer entre qualquer par de horários consecutivos num mesmo dia. Como há $|H| - 1$ pares por dia, isso é multiplicado pelo número de dias e pelo número de professores.

- Máximo de penalidade por alocação de aulas em dias não preferidos:

$$\text{max_non_preferred} = \max\left(1, \sum_{f \in F} |\{d \in D \mid d \notin \text{pref}_f\}|\right) \tag{26}$$

Para cada professor, é calculado o número de dias que não estão em sua lista de preferências, e esse total é somado para todos os professores.

- Máximo de penalidades por alocação de disciplinas obrigatórias no turno da tarde:

$$\text{max_afternoon} = \sum_{m \in M_{obr}} C_m \cdot H_t \tag{27}$$

Para cada disciplina obrigatória m, é considerado sua carga horária C_m como podendo ser alocada totalmente em todos os horários da tarde H_t. O valor total é o número máximo de *slots* que poderiam violar essa preferência.

- Máximo de penalidades por pré-requisitos alocados em dias diferentes:

$$\text{max_prereq} = \max\left(1, \sum_{ma \in M} \sum_{mb \in \text{prereq}(ma)} \sum_{d \in D} (\min(4, C_{ma}) + \min(4, C_{mb}))\right) \tag{28}$$

Para cada par de disciplina *ma* e seu pré-requisito *mb*, é calculado o número máximo de *slots* que podem ocorrer no mesmo dia. Como o modelo possui a restrição forte de que nenhuma disciplina pode ultrapassar 4 horários no mesmo dia, o cálculo é feito da seguinte maneira:

min(4, carga horária). Esse valor é somado para todos os dias possíveis, fornecendo dessa forma um limite superior por essa violação.

- Máximo de penalidades para disciplinas com alta taxa de reprovação alocadas no mesmo dia e horário:

$$\text{max_failure} = \max\left(1, \sum_{\substack{m_1,m_2 \in M_{txr} \\ m_1 \neq m_2, |p_{m_1} - p_{m_2}| = 1}} \sum_{d \in D} \sum_{h \in H} 1\right) \tag{29}$$

Conta-se a quantidade total de possíveis colisões de horário entre pares de disciplinas com alta taxa de reprovação que estão em semestres consecutivos. Cada combinação válida conta como uma possível violação.

Após o cálculo dos limites superiores para cada tipo de violação, é realizada a normalização de cada uma das penalidades descritas no modelo. Com isso, são obtidas as variáveis normalizadas:

- *prereq_penalty*: pré-requisitos de uma disciplina alocados em dias diferentes da mesma.
- $failure_penalty$: disciplinas com alta taxa de reprovação alocadas no mesmo dia e horário.
- *excess_days_penalty*: professores alocados em mais de 3 dias.
- *gap_penalty*: *gaps* entre aulas de um mesmo professor no mesmo dia.
- $pref_penalty$: alocação de professores em dias não preferidos.
- $afternoon_penalty$: alocação de disciplinas obrigatórias no turno da tarde.

Como o valor de cada uma dessas variáveis é calculado a partir da divisão entre o valor observado em uma solução específica e o valor máximo possível da violação, sua escala é padronizada entre 0 e 1. Esse processo é fundamental para permitir que todas as penalidades tenham uma influência justa e comparável na FO.

4.1.5 Função objetivo

Levando em consideração as restrições fortes, restrições fracas e normalização definidas previamente, a FO é determinada da seguinte forma:

$$\begin{aligned} \min\Big(& \alpha_1 \cdot \text{prereq_penalty} + \alpha_2 \cdot \text{failure_penalty} + \\ & \alpha_3 \cdot \text{excess_days_penalty} + \alpha_4 \cdot \text{gap_penalty} + \\ & \alpha_5 \cdot \text{pref_penalty} + \alpha_6 \cdot \text{afternoon_penalty}\Big) \end{aligned} \tag{30}$$

Onde cada α representa o valor da penalidade atribuído a cada restrição fraca.

Dessa forma, a FO é uma função ponderada que considera as penalidades associadas às restrições fracas do modelo proposto para o PAH, restrições essas que embora não inviabilizem uma solução, impactam significativamente na sua qualidade. A utilização dos coeficientes de penalidade α_i permitem uma atribuição diferente de pesos a cada restrição fraca considerada, de forma a priorizar as preferências institucionais e pedagógicas.

Ao minimizar essa função, se tem por propósito buscar uma solução viável que além de respeitar obrigatoriamente todas as restrições fortes, também atenda ao maior número possível de critérios desejáveis (restrições fracas), proporcionando dessa forma uma grade horária eficiente e que vai de encontro com as preferências humanas estabelecidas.

4.2 Ferramentas

Nesta subseção são apresentadas as ferramentas empregadas no desenvolvimento do sistema proposto para a geração de grades horárias. Cada ferramenta foi escolhida com base em sua adequação às necessidades do modelo. O *solver IBM ILOG CPLEX* foi utilizado devido à sua eficiência na resolução de problemas complexos de PLI. Por outro lado, o *Python* desempenha um papel fundamental no *back-end*, sendo responsável pelo tratamento de dados e comunicação com o *solver*. Já no desenvolvimento da interface de usuário, optou-se pelo uso do *TypeScript* em conjunto com o *React* para construção de uma aplicação moderna e responsiva, permitindo a interação com o usuário de maneira intuitiva.

4.2.1 *Solver IBM ILOG CPLEX*

Para implementação e resolução do modelo matemático proposto neste trabalho, o qual é baseado em PLI, optou-se pelo *solver* de otimização *IBM ILOG CPLEX*. A justificativa da escolha pelo *solver CPLEX* se deu não somente pela sua eficiência, mas também devido à sua ampla documentação, que facilita a implementação e ajustes do modelo, e juntamente a esses motivos está sua adoção em aplicações acadêmicas diversas, o que o torna uma ferramenta confiável e consolidada.

De acordo com IBM(25), o *solver* permite o desenvolvimento e a implementação rápida de modelos de otimização de decisões, utilizando programação matemática e por restrições. Ele tem por objetivo lidar com problemas complexos de tomada de decisão e oferecer um conjunto robusto de algoritmos que proporcionam eficiência na obtenção de soluções ótimas.

O *CPLEX* proporciona um desempenho computacional avançado, possui a capacidade de lidar com volumes consideráveis de variáveis e restrições e também oferece integração com diversas linguagens de programação e ambientes de modelagem matemática como, por exemplo, *Python* e *Java*. Além disso, o *solver* fornece um ambiente completo de modelagem e otimização, possibilitando a definição de funções objetivos e restrições de maneira estruturada, podendo-se trabalhar com modelos de diferentes tamanhos e complexidades, e ao mesmo tempo garantir que as soluções sejam encontradas

dentro de um tempo viável mesmo em problemas considerados computacionalmente difíceis, como o problema de otimização da grade de horários acadêmicos.

Somado aos motivos anteriormente citados, o *solver IBM ILOG CPLEX* fornece ao usuário um *log* detalhado durante a execução do modelo, contendo informações como número de nós explorados, cortes aplicados, tempo de execução, dentre outras informações. Esses dados permitem que o usuário realize a avaliação de como pequenas mudanças nos parâmetros definidos para o problema podem impactar na solução final, auxiliando dessa forma na tomada de decisões estratégicas, como a frequência em que o modelo aplicará heurísticas de solução, o nível de pré-processamento realizado dentro dos nós e o tempo máximo de execução. Juntamente a isso, ele também busca otimizar o tempo de execução ao implementar estratégias de cortes e heurísticas de busca local.

No contexto deste trabalho, a opção pela utilização do *solver CPLEX* se dá pela necessidade de resolução do problema de grade horária no Cefet *Campus* Petrópolis para o curso de Engenharia de Computação, para dessa forma garantir que os recursos disponíveis sejam utilizados de forma eficiente. Ao utilizar esta ferramenta, busca-se encontrar alocações ótimas para a grade horária acadêmica, que respeitem as restrições institucionais e andem de acordo com as necessidades específicas da instituição, minimizando conflitos de horário e oferecendo uma maior satisfação a docentes e discentes. Dado que este é um problema classificado como NP-difícil, a escolha de um *solver* eficiente como o *CPLEX* permite encontrar soluções viáveis em tempo hábil, para o tamanho de instância que será tratada nessa pesquisa.

Em suma, o *solver IBM ILOG CPLEX* se apresenta como uma ferramenta eficaz para a modelagem e resolução de problemas de otimização complexos, garantindo que as soluções sejam encontradas de maneira eficiente e dentro das restrições impostas pelo problema. De acordo com o objetivo desejado, ele busca minimizar ou maximizar funções objetivos, respeitando um conjunto de restrições lineares ou inteiras, buscando proporcionar soluções ótimas ou próximas ao ótimo. Ao oferecer alto desempenho, escalabilidade e flexibilidade, o *CPLEX* se mostrou uma ferramenta adequada para este modelo. Por ser um *solver* estabelecido, confiável e com boa documentação, a curva de aprendizado foi moderada o que viabilizou a implementação da solução de forma eficiente.

4.2.2 Tecnologias para o *Back-end*

O *Python* é uma linguagem de programação de alto nível, possui uma sintaxe simples o que proporciona uma ampla legibilidade. A linguagem possui uma grande difusão nos dias atuais devido à grande versatilidade e curva de aprendizado. Juntamente a essas características tem-se o fato do *Python* ser uma linguagem de programação interpretada, logo, ela não precisa passar pelo processo de compilação. As linguagens interpretadas possuem algumas vantagens como portabilidade, dado que o código pode ser executado em qualquer dispositivo que possua um interpretador instalado.

Adicionalmente, seu uso possibilita uma flexibilidade e dinamismo no desenvolvimento, visto que modificações podem ser realizadas no código sem que haja a necessidade de recompilá-lo. Porém

as linguagens interpretadas também apresentam algumas desvantagens, como seu desempenho inferior dado que o código é executado linha por linha e não há otimização por meio da compilação. Elas também consomem mais memória e CPU em comparação com linguagens compiladas e por isso não é o tipo de linguagem mais indicada para sistemas de alta performance.

No contexto deste trabalho, o *Python* será utilizado em conjunto com o *IBM ILOG CPLEX*, estando o código disponível publicamente no repositório do GitHub[9]. Essa escolha se dá pelas características citadas acima, assim como também pelo suporte à integração com o *CPLEX* por meio da biblioteca *docplex*[10]. Essa biblioteca proporciona uma interface de alto nível para modelagem de problemas de otimização linear, possibilitando que o usuário defina as restrições e FO de forma mais clara.

Entre outros motivos, se encontra a facilidade que o *Python* oferece na leitura e manipulação de dados em conjunto com bibliotecas como Pandas e Numpy. Essa característica se torna importante para a flexibilidade do sistema, dado que para este trabalho será necessário a leitura de informações referentes às disciplinas, disponibilidade de professores, horários, períodos, entre outros dados.

Por fim, o *Python* simplifica a modelagem e análise de dados, enquanto o *CPLEX* garante soluções de alta performance para problemas complexos, e com isso, obtém-se uma combinação adequada que possibilita a implementação ágil de modelos matemáticos e uma análise detalhada dos resultados. Dessa maneira o uso do *Python* nesse trabalho não se limita a apenas um meio de implementação, mas a uma linguagem que contribuirá para o desenvolvimento de uma solução eficiente e clara.

4.2.3 Tecnologias para o *Front-end*

Com a finalidade de tornar o sistema acessível, interativo e fácil de ser utilizado, foi desenvolvido uma aplicação *front-end*. A aplicação permite a inserção de informações necessárias para a geração da grade horária, assim como uma revisão detalhada das mesmas.

O objetivo principal da aplicação é possibilitar uma visualização clara e organizada da solução encontrada pelo modelo de otimização, permitindo aos usuários a compreensão e interpretação de como os horários foram alocados, possibilitando dessa maneira uma análise qualitativa e quantitativa do resultado.

Por meio dela também é possível exportar, em arquivo PDF, a grade horária de todos os semestres gerada pelo modelo, facilitando dessa forma a comparação entre diferentes soluções obtidas, assim como também o compartilhamento dessas informações.

Para o desenvolvimento da aplicação *front-end*, responsável por auxiliar o usuário na criação e avaliação da grade horária, optou-se pela utilização do *TypeScript* em conjunto com o *framework React*, tecnologias que são capazes de oferecer uma aplicação moderna para construção de interfaces interativas.

[9] https://github.com/gabimusants/cefet-timetable-optimization
[10] https://pypi.org/project/docplex/

O *TypeScript* é uma linguagem de programação fortemente tipada construída em cima do *JavaScript*. Devido a tipagem, o *TypeScript* se torna uma linguagem que permite uma maior previsibilidade do comportamento da aplicação, assim como facilita a identificação de erros durante o desenvolvimento e melhora de maneira significativa a manutenção do sistema. A tipagem também contribui para uma melhor documentação do código, o tornando mais legível, compreensível e confiável.

Por sua vez, o *React*, *framework* utilizado em conjunto com o *TypeScript*, é bastante popular devido a sua performance, eficiência e flexibilidade no desenvolvimento de aplicações web interativas, e em especial aquelas que necessitam de atualização frequente de dados em tempo real ou interfaces muito dinâmicas. A estrutura do *framework* é baseada em componentes reutilizáveis, e essa característica facilita a organização e manutenção do código. Cada componente caracteriza uma parte isolada da interface, e eles podem ser reutilizados em conjunto com outros componentes para formar telas completas. Essa perspectiva de módulos ajuda na escalabilidade do projeto, pois permite que novos recursos sejam adicionados ao sistema de maneira que não comprometa o funcionamento de partes que já estão desenvolvidas.

Outra característica importante do *React* é sua renderização reativa, o que significa que sempre que os dados da aplicação sofrem uma alteração, os componentes responsáveis por exibí-los são atualizados de forma automática. Isso se torna um ponto bastante útil na construção deste trabalho, visto que as soluções de alocação podem variar a cada execução do modelo *CPLEX* considerando diferentes dados de entrada, e a utilização do *framework React* garantirá que cada alteração na solução do PAH refletirá automaticamente na interface.

Além dos pontos citados anteriormente, o *React* também possui uma ampla opção de bibliotecas que podem ser integradas de forma a aprimorar a experiência do usuário. No contexto deste trabalho, a biblioteca *React Router*[11] possibilitará uma navegação fluida entre páginas. O *framework* também proporcionará a integração entre o *front-end* e o *back-end*, permitindo que as soluções geradas pelo *CPLEX* sejam requisitadas via *API (Application Programming Interface)*, recebidas e renderizadas na tela de forma organizada e compreensível.

A escolha por aplicar o *TypeScript* em conjunto com o *React* se baseia na conexão entre as duas tecnologias. Quando combinado com o *TypeScript*, os componentes *React* podem ser precisamente definidos quanto às suas propriedades, estados internos e eventos, evitando assim ambiguidades e possibilitando uma estrutura de código mais eficiente e escalável.

A combinação entre *Python*, *CPLEX*, *TypeScript* e *React* resulta em uma arquitetura sólida, moderna e eficiente, sendo assim capaz de proporcionar uma experiência responsiva e eficaz para o usuário final, desta maneira se alinhando aos objetivos deste trabalho.

[11] https://reactrouter.com

4.3 Desenvolvimento da Interface do Usuário

O sistema que foi desenvolvido para a geração de grades horárias para o curso de Engenharia de Computação do Cefet/RJ *Campus* Petrópolis possui uma interface simples e objetiva, dividida em três telas principais para interação com o usuário.

- Tela inicial.
- Tela para *upload* de dados.
- Tela para visualização e exportação dos resultados.

Cada tela tem por objetivo cumprir um papel específico no fluxo de uso, dessa forma, proporcionando uma experiência fluida para o usuário, desde o envio dos dados até a visualização da grade horária otimizada gerada.

Ao acessar a aplicação, o usuário visualizará a tela inicial, apresentada na Figura 23, que possui um botão cuja função principal é redirecioná-lo para a etapa seguinte do processo. Essa tela tem por objetivo servir como um ponto de entrada, permitindo que o usuário entenda que se trata de uma aplicação voltada para a otimização e planejamento de horários acadêmicos.

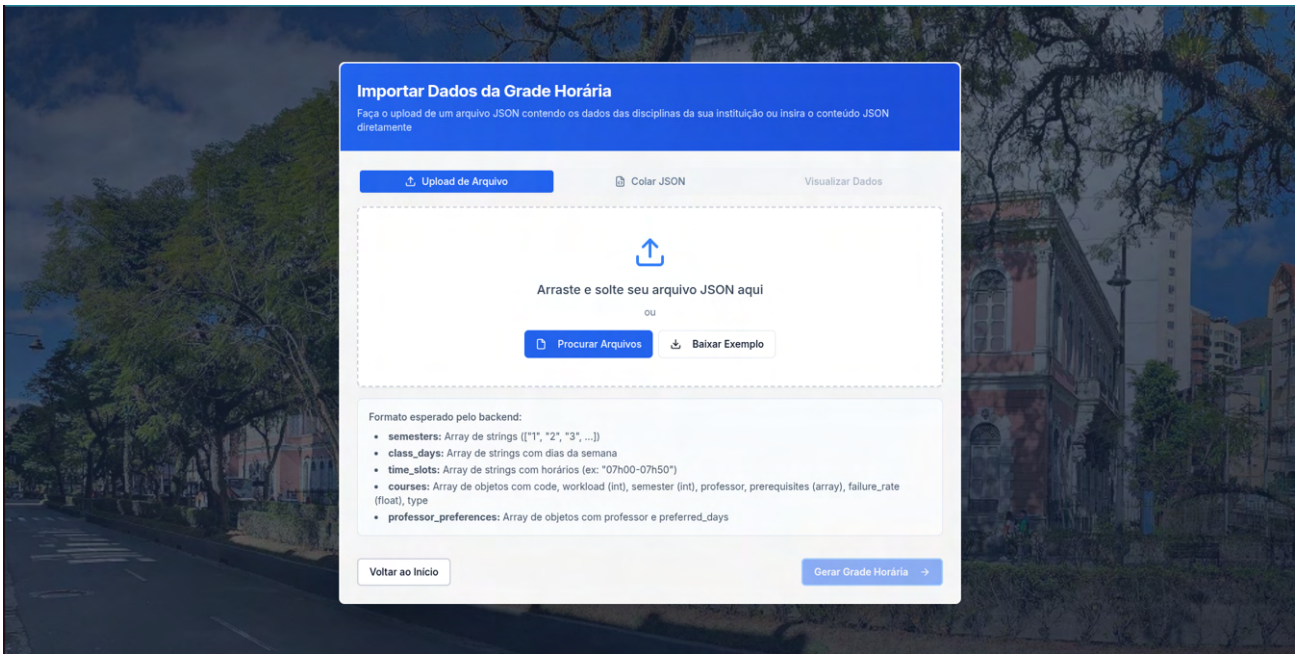
Figura 3: Tela inicial do sistema.



Na tela principal para *upload* de dados, o usuário tem à disposição três opções principais:

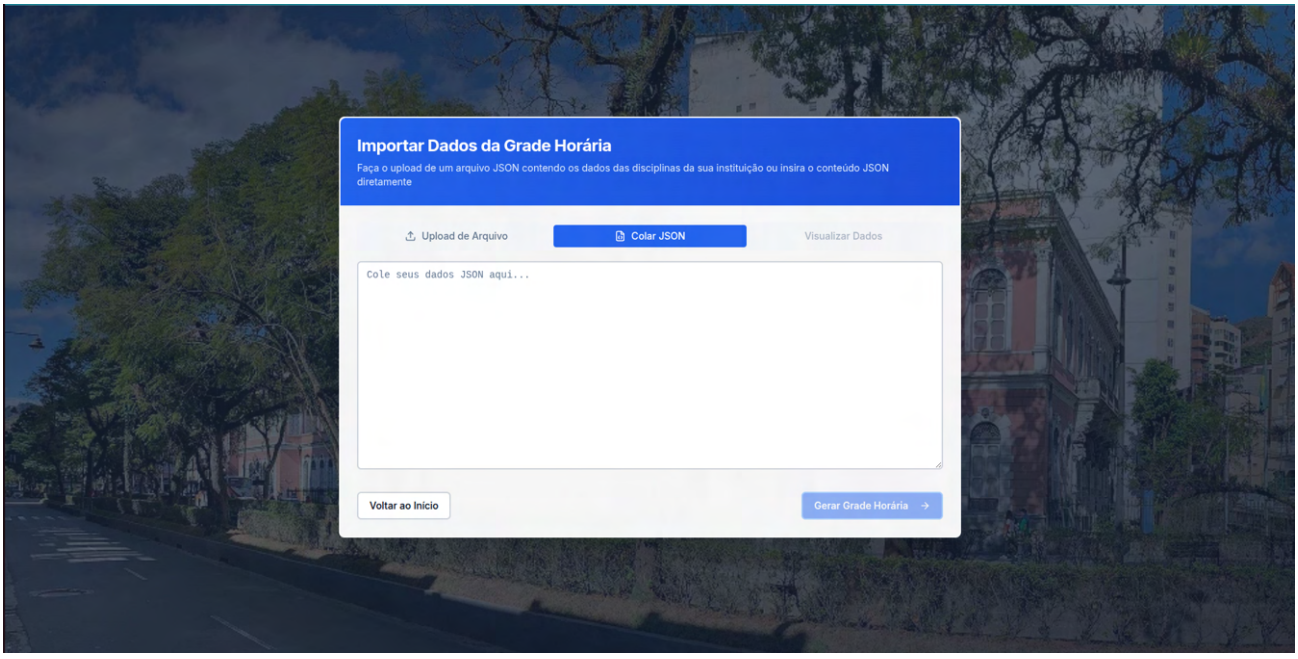
- Fazer o *upload* de um arquivo no formato *JSON* com parâmetros requeridos pelo modelo matemático, isto é, os dados necessários para geração da grade horária (professores, disciplinas, dias letivos, horários etc.), como é possível observar no Apêndice 1. (Figura 4)

Figura 4: Upload do arquivo com dados de entrada do modelo.



- Inserir diretamente o conteúdo no formato *JSON* no campo especificado para tal função. (Figura 5)

Figura 5: Inserção direta de dados.



- Visualizar de forma resumida e clara os dados inseridos a partir do *JSON*, como:
  - Resumo geral (Figura 6)

Figura 6: Resumo geral dos dados carregados.

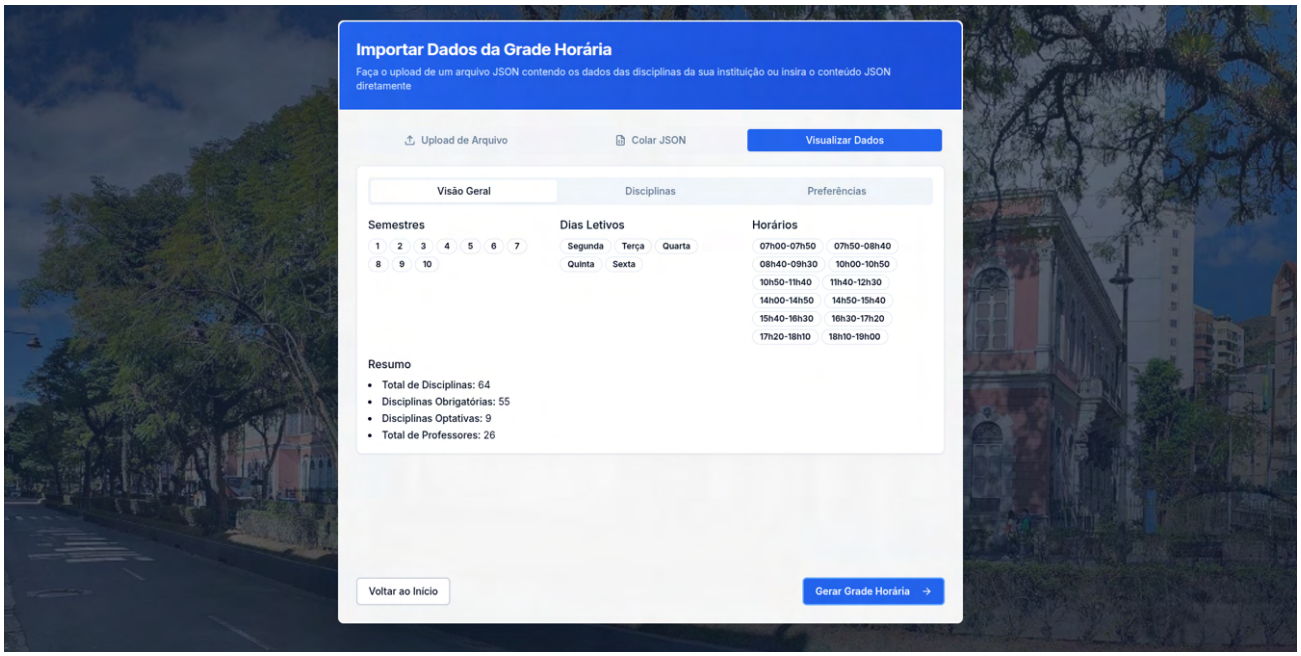
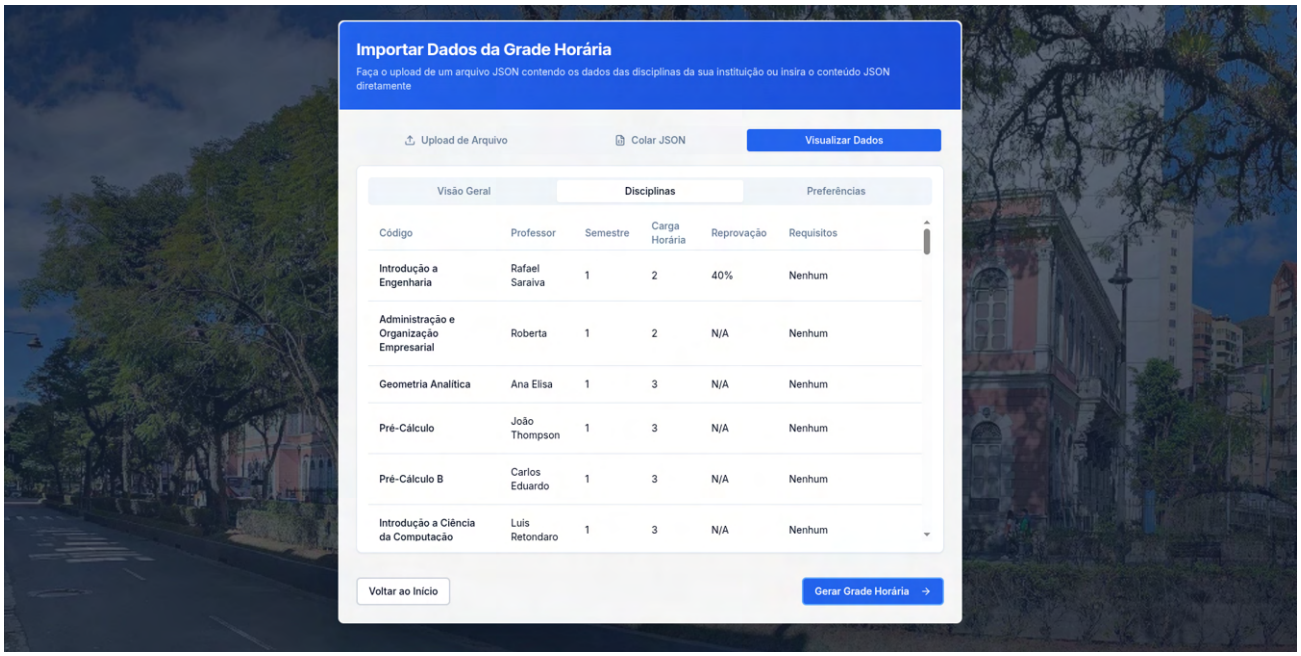


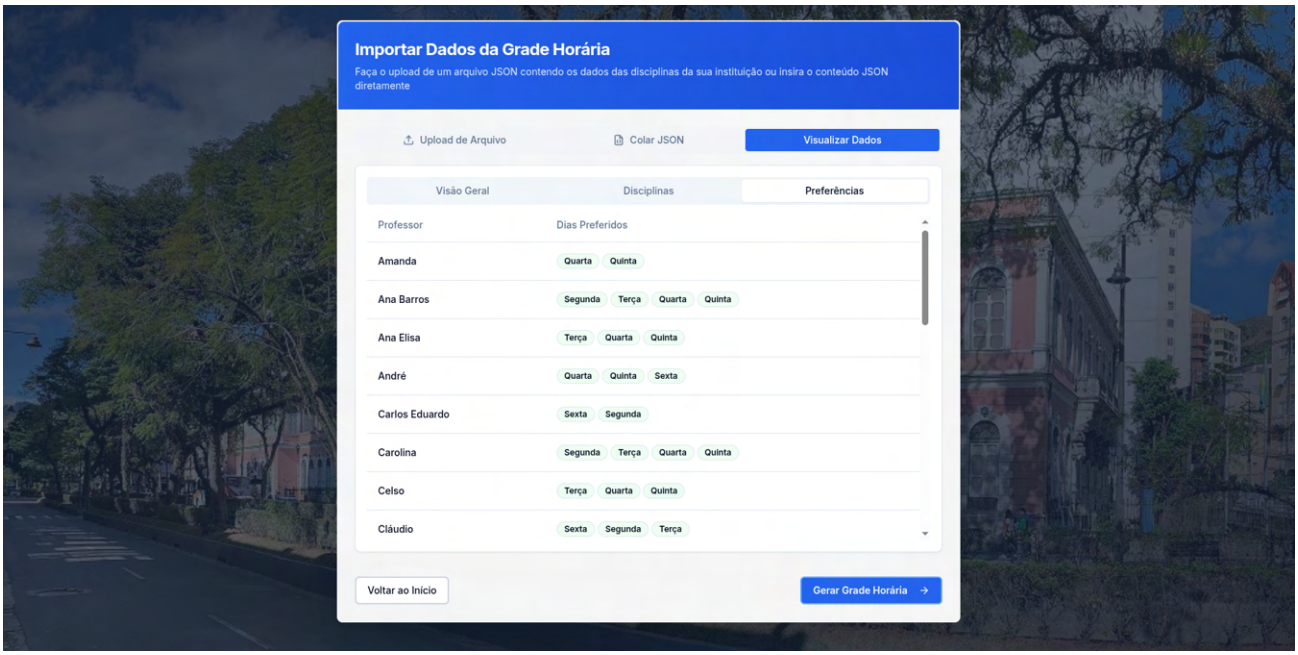
Figura 7: Disciplinas e informações.



- Disciplinas e suas informações (Figura 7)
- Preferências dos dias letivos de professores (Figura 8)

Também há a possibilidade de baixar um exemplo de arquivo *JSON*, que ajuda o usuário a entender o formato de entrada esperado pelo *back-end*. O sistema espera um conjunto de estruturas específicas no arquivo *JSON*, como:

Figura 8: Preferências dos dias letivos de professores.

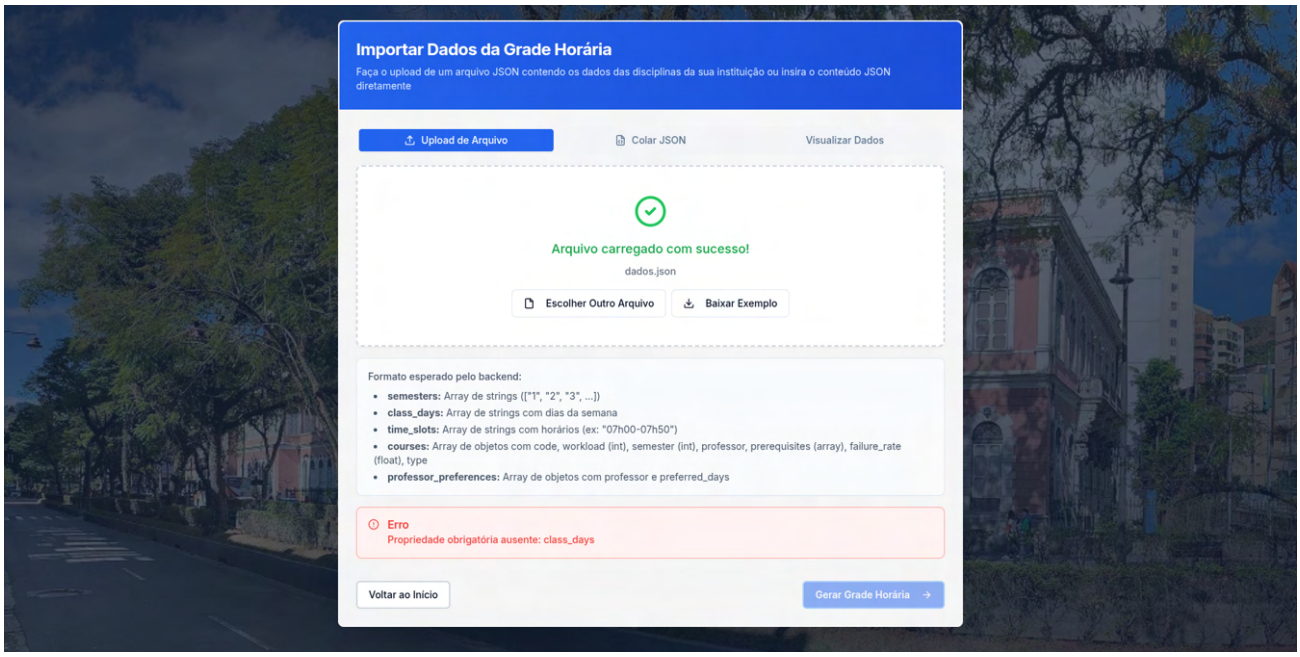


- *Array* de semestres
- Dias letivos
- Horários disponíveis para alocação de aulas
- Lista de preferências dos dias letivos de cada professor, contendo:
  - Nome do professor
  - *Array* de dias letivos preferidos pelo professor
- Lista de disciplinas com informações detalhadas sobre as mesmas, como:
  - Nome
  - Carga horária
  - Semestre
  - Professor
  - Taxa de reprovação (opcional)
  - *Array* de pré-requisitos (caso exista)
  - Indicação se a disciplina é obrigatória ou optativa.

O sistema realiza a verificação da consistência dos dados fornecidos pelo usuário, seja por meio da inserção de um arquivo *JSON* ou pela inserção direta na seção “Colar *JSON*”. Essa etapa de

validação é responsável por identificar dados obrigatórios ausentes ou estruturas incompatíveis com o esperado pelo *back-end*. Quando algum problema é detectado, o sistema impede a execução e exibe uma mensagem de erro informando qual dado está ausente ou incorreto, o que pode ser observado na Figura 9. A partir dessa mensagem, o usuário pode corrigir os dados de duas formas: *(i)* editando diretamente o arquivo *JSON* e reenviando-o ao sistema, ou *(ii)* utilizando a seção "Colar *JSON*" para ajustar o conteúdo manualmente.

Figura 9: Mensagem de erro devido a arquivo *JSON* com dados obrigatórios ausentes.



Após a inserção dos dados no sistema, o usuário deverá clicar no botão "Gerar Grade Horária", que pode ser visto na Figura 4, o qual aciona a execução do modelo PLI no *back-end*. O *back-end* processa os dados inseridos, levando em consideração diversas restrições e preferências, e então retorna uma solução ótima. Essa solução é então exibida ao usuário na tela de resultados, onde é possível visualizar a grade horária gerada de forma objetiva e interativa.

Na tela de resultados, o sistema apresenta uma visão semanal da grade horária gerada, com os dias da semana dispostos em colunas e os horários disponíveis em linhas, como pode ser observado na Figura 10. Cada célula da tabela mostra as disciplinas alocadas, juntamente dos nomes dos professores responsáveis, utilizando cores diferentes de maneira a facilitar a leitura e análise visual. Além disso, há um seletor no canto superior direito que permite filtrar a visualização por períodos específicos. No menu superior da tela, também se encontra uma opção para visualização diária da grade horária criada, o que permite ao usuário uma análise mais detalhada e específica do horário das disciplinas, acompanhado de seus respectivos professores, o que pode ser observado na Figura 11.

Localizado na parte inferior da tela, encontram-se os botões que permitem baixar os dados utilizados na entrada, salvar a solução gerada em formato *JSON*, ou ainda exportar a grade horária como *PDF* (Figura 12), para fins de documentação ou compartilhamento. Também existe a opção de

Figura 10: Grade horária com filtro semanal.

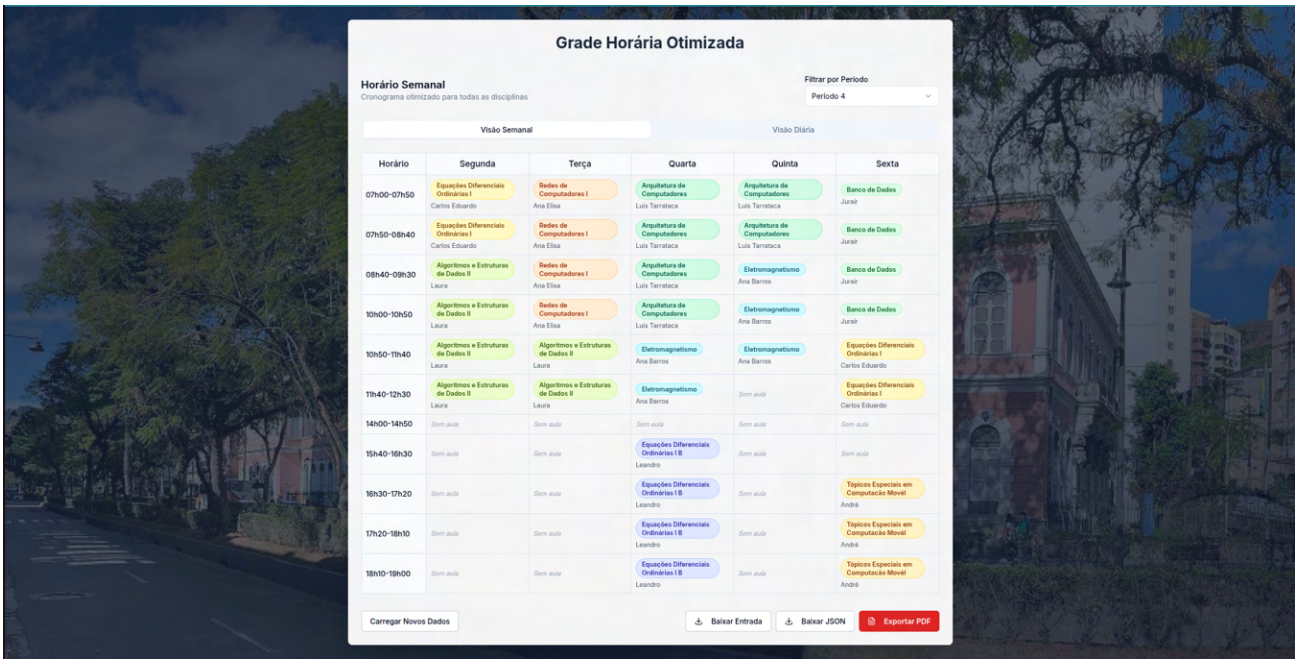
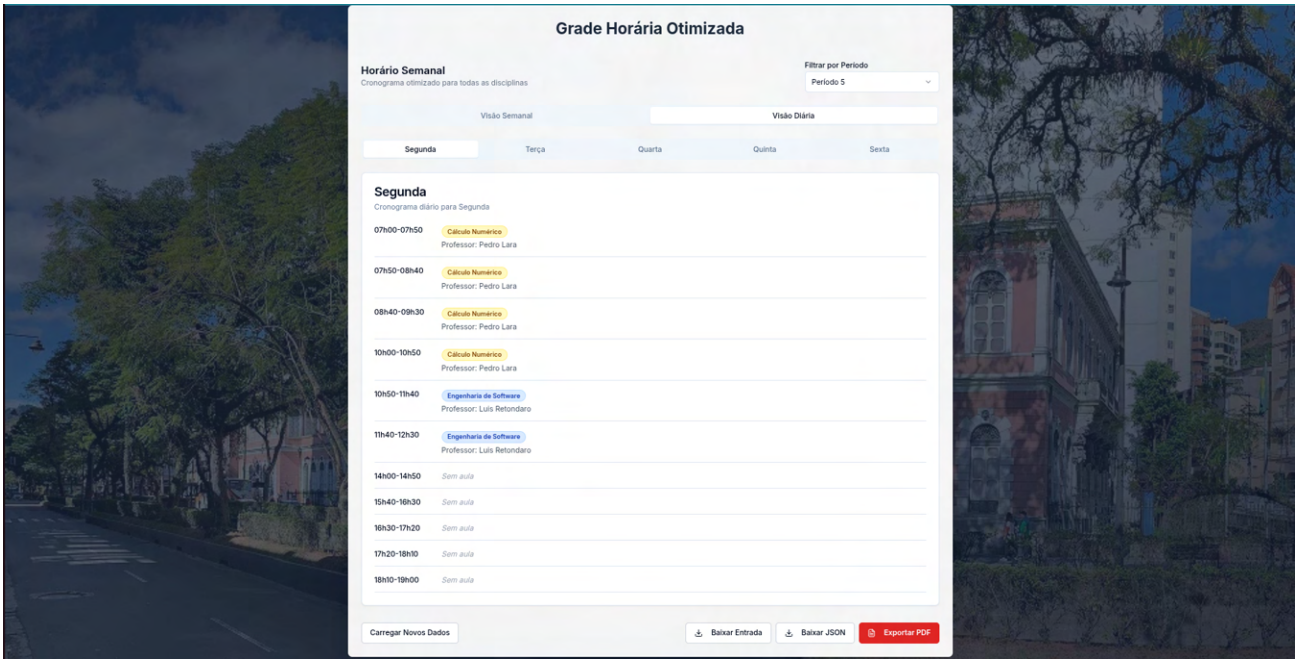


Figura 11: Grade horária com filtro diário.

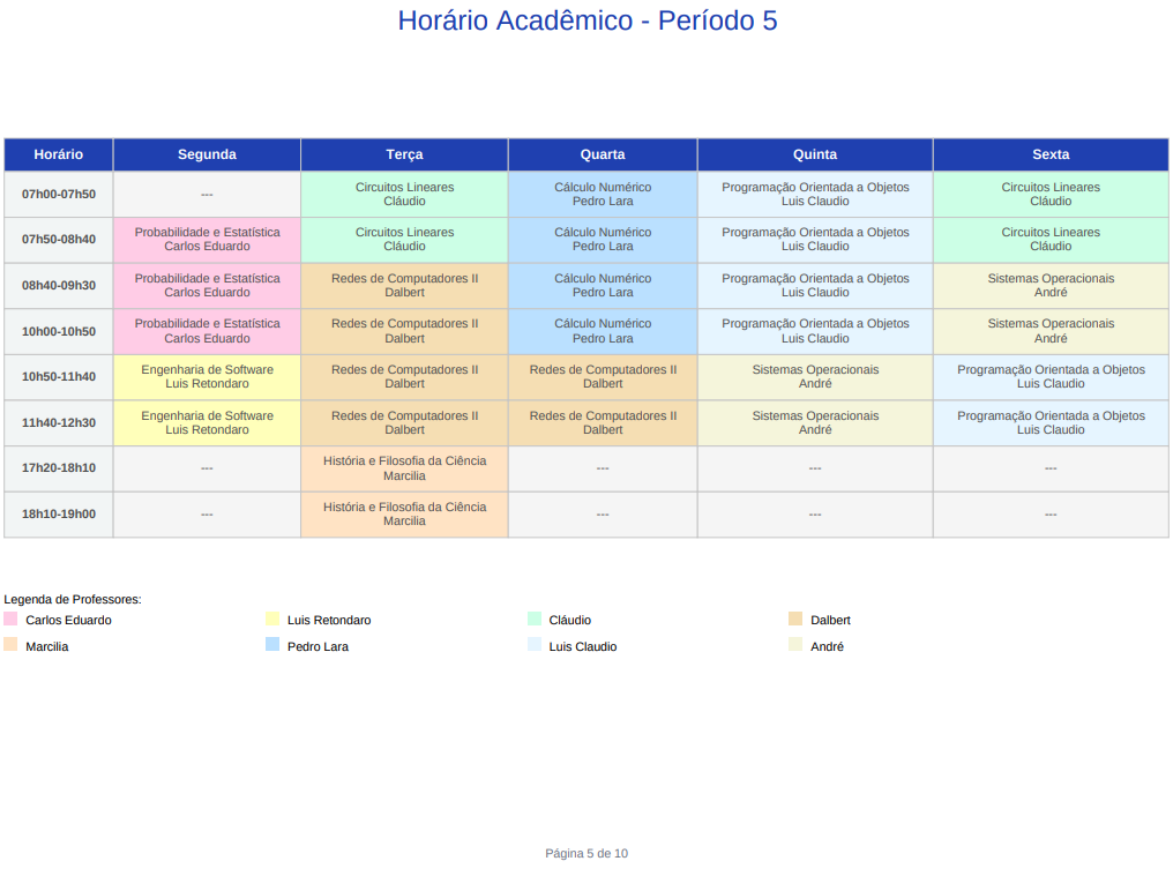


"Carregar Novos Dados", que permite ao usuário a reinicialização do processo, retornando à tela de *upload* de dados, para gerar uma nova grade com dados diferentes.

O fluxograma da Figura 13 representa graficamente o funcionamento do sistema, mostrando de forma sequencial e lógica como o usuário interage com a aplicação, desde o início até a visualização dos resultados. O modelo apresenta os seguintes passos:

1. Usuário começa na tela inicial.

Figura 12: Grade horária - arquivo *PDF*.



Horário Acadêmico - Período 5

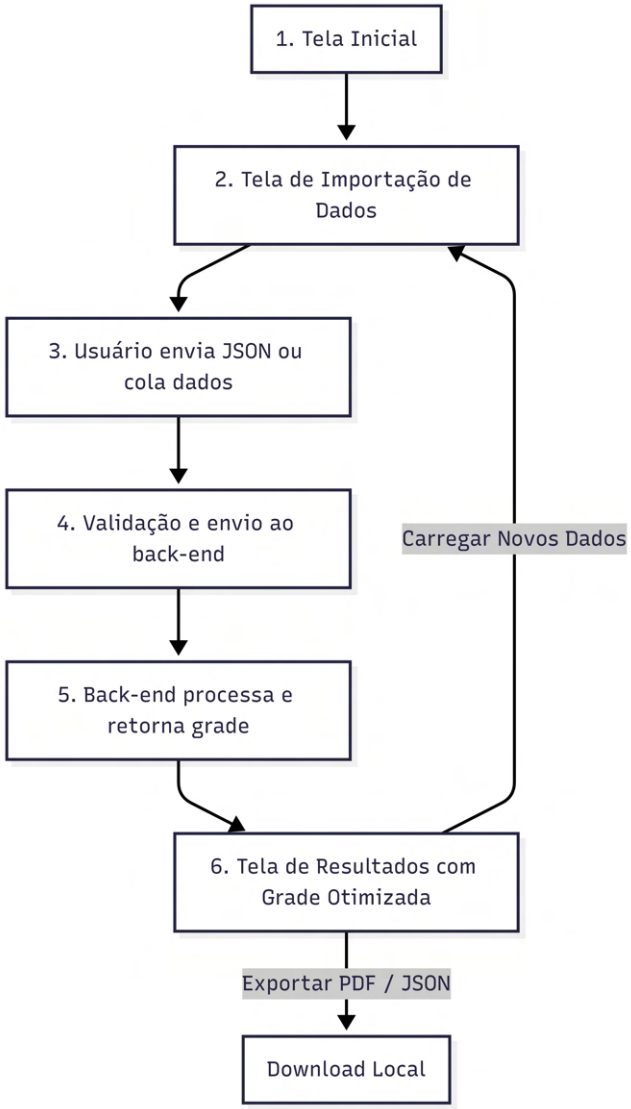
Horário	Segunda	Terça	Quarta	Quinta	Sexta
07h00-07h50	---	Circuitos Lineares Cláudio	Cálculo Numérico Pedro Lara	Programação Orientada a Objetos Luis Claudio	Circuitos Lineares Cláudio
07h50-08h40	Probabilidade e Estatística Carlos Eduardo	Circuitos Lineares Cláudio	Cálculo Numérico Pedro Lara	Programação Orientada a Objetos Luis Claudio	Circuitos Lineares Cláudio
08h40-09h30	Probabilidade e Estatística Carlos Eduardo	Redes de Computadores II Dalbert	Cálculo Numérico Pedro Lara	Programação Orientada a Objetos Luis Claudio	Sistemas Operacionais André
10h00-10h50	Probabilidade e Estatística Carlos Eduardo	Redes de Computadores II Dalbert	Cálculo Numérico Pedro Lara	Programação Orientada a Objetos Luis Claudio	Sistemas Operacionais André
10h50-11h40	Engenharia de Software Luis Retondaro	Redes de Computadores II Dalbert	Redes de Computadores II Dalbert	Sistemas Operacionais André	Programação Orientada a Objetos Luis Claudio
11h40-12h30	Engenharia de Software Luis Retondaro	Redes de Computadores II Dalbert	Redes de Computadores II Dalbert	Sistemas Operacionais André	Programação Orientada a Objetos Luis Claudio
17h20-18h10	---	História e Filosofia da Ciência Marcilia	---	---	---
18h10-19h00	---	História e Filosofia da Ciência Marcilia	---	---	---

Legenda de Professores:
Carlos Eduardo
Luis Retondaro
Cláudio
Dalbert
Marcilia
Pedro Lara
Luis Claudio
André

Página 5 de 10

2. É levado para a tela de carregamento dos dados.
3. Insere os dados via *upload* de arquivo ou inserção manual, ambos no formato *JSON*.
4. Validação e envio dos dados para o *back-end*.
5. O *back-end* processa os dados, executa o modelo PLI, e gera a grade de horários otimizada.
6. O resultado (grade gerada) é apresentada na tela de resultados.
7. A partir da tela de resultados o usuário pode tanto exportar a grade horária gerada (Apêndice 2), como retornar ao *upload* para modificar os dados e gerar uma nova solução.

Figura 13: Fluxograma do sistema de geração de grade horária otimizada.



5 Resultados

A partir do sistema proposto (modelo PLI e aplicação), foi possível gerar uma solução ótima para o problema de determinação de grades horárias otimizadas para cursos de graduação. Os testes para validação do sistema foram realizados utilizando as informações do curso de Engenharia de Computação do Cefet/RJ - *Campus* Petrópolis. O modelo foi implementado em *Python* com a biblioteca *Docplex* e resolvido por meio do *solver CPLEX*, considerando tanto as restrições fortes quanto as restrições fracas (Capítulo 4).

O tempo médio de resolução foi de 3 minutos e 28 segundos. A solução encontrada respeitou todas as restrições fortes impostas, garantindo a viabilidade acadêmica da grade horária. Os testes foram realizados em um computador com sistema operacional Linux, processador AMD Ryzen 5 3600 e 16GB de memória RAM.

As informações utilizadas para realização dos testes foram fornecidas pelo coordenador do curso e também obtidas por meio do site do curso[12], sendo o problema determinado por:

- 10 períodos;
- 26 professores;
- 55 disciplinas obrigatórias;
- 9 disciplinas optativas;
- 6 *slots* de tempo disponíveis para aulas na parte da manhã;
- 6 *slots* de tempo disponíveis para aulas na parte da tarde.

A Tabela 2 apresenta os pesos atribuídos a cada uma das restrições fracas consideradas nos experimentos. Esses valores foram utilizados de forma uniforme em todas as execuções, permitindo assim uma avaliação comparável entre os diferentes cenários testados.

Tabela 2: Pesos atribuídos às restrições fracas nos experimentos.

Restrição Fraca	**Peso**
Pré-requisitos em dias distintos	0,05
Gaps de aulas no mesmo dia	0,25
Conflito entre disciplinas de alta reprovação	0,18
Excesso de dias por professor	0,05
Alocação em dias não preferenciais	0,02
Disciplinas obrigatórias à tarde	0,45

Foram realizados testes alterando a quantidade de matérias sinalizadas com altas taxas de reprovação e dentre os principais indicadores obtidos a partir das soluções estão:

[12] https://e-computacao.com.br/

Tabela 3: Resumo das penalidades associadas às restrições fracas com 10 matérias críticas.

Restrição Fraca	**Ocorrências**
Pré-requisitos em dias distintos	49
Gaps de aulas no mesmo dia	44
Conflito entre disciplinas de alta reprovação	0
Excesso de dias por professor	0
Alocação em dias não preferenciais	0
Disciplinas obrigatórias à tarde	0
Tempo de execução	**3 minutos e 42 segundos**

Tabela 4: Resumo das penalidades associadas às restrições fracas com 13 matérias críticas.

Restrição Fraca	**Ocorrências**
Pré-requisitos em dias distintos	50
Gaps de aulas no mesmo dia	43
Conflito entre disciplinas de alta reprovação	0
Excesso de dias por professor	0
Alocação em dias não preferenciais	0
Disciplinas obrigatórias à tarde	0
Tempo de execução	**2 minutos e 46 segundos**

Tabela 5: Resumo das penalidades associadas às restrições fracas com 15 matérias críticas.

Restrição Fraca	**Ocorrências**
Pré-requisitos em dias distintos	49
Gaps de aulas no mesmo dia	43
Conflito entre disciplinas de alta reprovação	0
Excesso de dias por professor	0
Alocação em dias não preferenciais	0
Disciplinas obrigatórias à tarde	0
Tempo de execução	**3 minutos e 22 segundos**

Com base nos resultados apresentados na Tabela 3, é possível observar que a violação de pré-requisitos se destacou como a principal fonte de penalidade com 49 ocorrências. Apesar de seu peso relativo na função ser moderado, seu impacto global é significativo, o que indica que há um número elevado de disciplinas e seus respectivos pré-requisitos sendo alocados em dias diferentes, contrariando a intenção de agrupá-las afim de evitar possíveis choques de horários entre as disciplinas que podem ser cursadas simultaneamente.

Os *gaps* de aulas tiveram 44 ocorrências, já as demais restrições não geraram qualquer penalidade nestes cenários. Esse resultado indica que o modelo atendeu de forma eficaz às restrições fracas, fornecendo solução adequada às preferências dos docentes e à distribuição equilibrada da carga horária.

Analisando os resultados exibidos nas tabelas 3, 4, 5 é possível observar que o aumento no número de matérias com alta taxa de reprovação não tem grande impacto na solução, isto é, interfere pouco na quantidade de violações das duas primeiras restrições fracas listadas nessas tabelas e mantém o atendimento das demais, assim como também não há grande impacto no tempo de execução. Tal comportamento mostra que o modelo lida bem com o aumento de complexidade, mantendo o controle sobre os principais fatores de penalidade.

Além da qualidade dos resultados obtidos, se encontra o ganho de eficiência proporcionado pela modelagem automatizada. O modelo de otimização foi capaz de gerar uma solução viável em aproximadamente 3 minutos, em contraste com o processo tradicional de formulação das grades horárias, que pode demandar, de acordo com informações proporcionadas pelo orientador do curso, entre 10 a 20 horas para formulação de uma versão, podendo ser necessário a construção de até 8 versões, o que demandaria um tempo investido bastante elevado. Assim, a abordagem automatizada deste trabalho não apenas reduz drasticamente o esforço humano, mas também permite explorar de forma mais rápida diferentes cenários, realizar ajustes finos e garantir maior eficiência e aderência relacionadas a versão final do horário gerado.

Outro benefício relevante do modelo é a diminuição das dificuldades atreladas à gestão de relações humanas, levando em consideração os pedidos de alterações posteriores por parte dos docentes. No processo manual, esses pedidos geram uma reestruturação de horários pré-estabelecidos, o que pode comprometer a factibilidade e coerência da grade horária, e demandar tempo de retrabalho devido o encadeamento de alterações dado um pedido de mudança. Com a abordagem proposta, modificações como preferência de dias podem ser ajustadas por meio da edição do conteúdo do arquivo de entrada. Após alteração, seria necessário executar novamente o modelo para geração de uma nova solução otimizada, que respeite as novas preferências impostas, proporcionando assim agilidade, flexibilidade e maior controle sobre o processo de construção da grade horária.

6 Considerações Finais

Neste trabalho, foi desenvolvido e apresentado um sistema de geração de grades horárias otimizadas, aplicado ao contexto do Centro Federal de Educação Tecnológica Celso Suckow da Fonseca - *Campus* Petrópolis, com potencial de uso em diversos cursos de graduação. Foi proposto um modelo de PLI, e utilizado o *solver CPLEX* em conjunto com *Python* e a biblioteca *docplex* para o desenvolvimento do *back-end*, assim como *TypeScript* e *React* para o desenvolvimento do *front-end*.

O problema de alocação de horários conhecido pela sua complexidade combinatória foi abordado por meio da construção de um modelo matemático com restrições fortes e fracas. As restrições fortes representam condições que obrigatoriamente devem ser atendidas pela solução, enquanto as restrições fracas correspondem a preferências ou critérios desejáveis, cuja violação não inviabiliza a solução, mas é penalizada na função objetivo. A metodologia utilizada se mostrou eficiente para a adaptabilidade do contexto da instituição e a utilização de uma função objetivo composta por termos normalizados garantiu o balanceamento entre as diferentes restrições fracas.

Os testes foram realizados com dados reais do curso de Engenharia de Computação do Cefet/RJ *Campus* Petrópolis. Foram consideradas as disciplinas obrigatórias e optativas, os professores habilitados para cada uma delas, os horários e dias disponíveis, as preferências dos docentes e demais restrições institucionais. Esses dados permitiram simular cenários realistas e avaliar a capacidade do modelo ao lidar com as particularidades da instituição.

Os resultados obtidos mostram que o modelo é capaz de produzir grades horárias viáveis, respeitando todas as restrições obrigatórias e minimizando as violações das restrições que não inviabilizam o horário gerado. Outra característica importante foi a eficiência temporal do modelo. Seu tempo médio de execução foi de aproximadamente 3 minutos, sendo significativamente mais rápido do que a construção manual que pode variar entre 10 a 20 horas.

Além disso, o modelo possui a capacidade de adaptação a mudanças humanas e operacionais, como pedidos de alteração de horários por parte de docentes. Esses ajustes podem ser facilmente modificados por meio da edição do arquivo de entrada do modelo, sendo acompanhado de uma nova execução do modelo.

Como trabalho futuro, pode-se citar a análise de expansão da metodologia para incluir o ensalamento ótimo dos cursos, assim como a aplicação de técnicas de *machine learning* para estimativa dos pesos da penalidade, juntamente da realização de pré-processamentos que reduzam o espaço de busca e acelerem a convergência do modelo. Outro aspecto que pode ser explorado são técnicas de otimização multiobjetivo, na tentativa de produzir um conjunto de soluções ótimas que podem ser fornecidas para o tomador de decisão.

Em suma, o uso da Programação Linear Inteira se mostrou uma solução robusta e promissora para a construção automática de grades de horário, proporcionando ganhos em eficiência, escalabilidade e qualidade na criação dos horários. A aplicabilidade do modelo foi validada com dados reais do curso de Engenharia de Computação do Cefet/RJ - *Campus* Petrópolis.

REFERÊNCIAS

1 PEREIRA, V. et al. Linear integer model for the course timetabling problem of a faculty in rio de janeiro. **Advances in Operations Research**, 2016.

2 HILLIER, F. S. **Introduction to Operations Research**. San Francisco,: Holden-Day, 1967.

3 ANDRADE, E. de. **Introdução à Pesquisa Operacional - Métodos e Modelos para Análise de Decisões**. LTC, 2015. ISBN 9788521629429. Disponível em: https://books.google.com.br/books?id=vsH6zwEACAAJ.

4 DASKALAKI, S.; BIRBAS, T.; HOUSOS, E. An integer programming formulation for a case study in university timetabling. **European Journal of Operational Research**, v. 153, n. 1, p. 117–135, 2004. ISSN 0377-2217. Timetabling and Rostering. Disponível em: https://www.sciencedirect.com/science/article/pii/S0377221703001036.

5 EL-SHERBINY, M. M. et al. Genetic algorithm for solving course timetable problems. **International Journal of Computer Applications**, 2015.

6 VIEIRA, F.; MACEDO, H. Sistema de alocação de horários de cursos universitários: Um estudo de caso no departamento de computação da universidade federal de sergipe. **Scientia Plena**, v. 7, n. 3, abr. 2011. Disponível em: https://www.scientiaplena.org.br/sp/article/view/163.

7 ALGHAMDI, H. et al. A review of optimization algorithms for university timetable scheduling. **Engineering, Technology Applied Science Research**, 2020.

8 WOLSEY, L. **Integer Programming**. Wiley, 1998. (Wiley Series in Discrete Mathematics and Optimization). ISBN 9780471283669. Disponível em: https://books.google.com.br/books?id=x7RvQgAACAAJ.

9 Gurobi Optimization, LLC. **Gurobi Optimizer**. 2025. Disponível em: https://www.gurobi.com/solutions/gurobi-optimizer/.

10 PHILLIPS, A. E. et al. Integer programming for minimal perturbation problems in university course timetabling. **Annals of Operations Research**, 2017.

11 HERTZ, A. Tabu search for large scale timetabling problems. **European Journal of Operational Research**, v. 54, n. 1, p. 39–47, 1991. ISSN 0377-2217. Disponível em: https://www.sciencedirect.com/science/article/pii/037722179190321L.

12 CATALDO, A. et al. An integer programming approach to curriculum-based examination timetabling. **Annals of Operations Research**, 2017.

13 GANGULI, R.; ROY, S. D. A study on course timetable scheduling using graph coloring approach. **International Journal of Computational and Applied Mathematics.**, 2017. Disponível em: https://api.semanticscholar.org/CorpusID:195747028.

14 HAMBALI, M.; OLASUPO, Y.; DALHATU, M. Automated university lecture timetable using heuristic approach. **Nigerian Journal of Technology**, v. 39, p. 1–14, 04 2020.

15 DANIEL, P.; ALIMI, M.; MODI, B. Paperless master timetable scheduling system. **International Journal of Applied Science and Technology**, v. 8, 01 2018.

16 KAHAR, M. N. M.; KENDALL, G. A great deluge algorithm for a real-world examination timetabling problem. **Journal of the Operational Research Society**, Taylor & Francis, v. 66, n. 1, p. 116–133, 2015.

17 ABRAMSON, D.; ABELA, J. **A parallel genetic algorithm for solving the school timetabling problem**. [S.l.]: Citeseer, 1991.

18 WOHLIN, C. et al. Successful combination of database search and snowballing for identification of primary studies in systematic literature studies. **Information and Software Technology**, v. 147, p. 1–12, 2022. ISSN 0950-5849. Disponível em: https://doi.org/10.1016/j.infsof.2022.106908.

19 QUEIROZ, R. S. et al. Proposta de um modelo para o school timetabling problem em um instituto federal de educaÇÃo, ciÊncia e tecnologia. **Anais do Simpósio Brasileiro de Pesquisa Operacional**, 2020.

20 BUCCO, G. B.; BORNIA-POULSEN, C. J.; BANDEIRA, D. L. Development of a linear programming model for the university course timetabling problem. 2017.

21 SCHIMMELPFENG, K. et al. Application of a real-world university-course timetabling model solved by integer programming. **OR Spectrum**, 2007.

22 GALL, A. F. N.; SILVA, A. R. V. da. Alocação de horários de professores e turmas em um curso universitário. **Simpósio Brasileiro de Pesquisa Operacional**, 2016. Disponível em: http://www.din.uem.br/sbpo/sbpo2016/pdf/155618.pdf.

23 MIRHASSANI, S. A. A computational approach to enhancing course timetabling with integer programming. **Applied Mathematics and Computation**, 2006.

24 AKKOYUNLU, E. A. A linear algorithm for computing the optimum university timetable. **The Computer Journal**, 1973.

25 IBM. **IBM ILOG CPLEX Optimization Studio**. 2025. Disponível em: https://www.ibm.com/products/ilog-cplex-optimization-studio.

Apêndice A - Arquivo de entrada *JSON*

```
{
  "semesters": ["1", "2", "3", "4", "5", "6", "7", "8", "9", "10"],
  "class_days": ["Segunda", "Terça", "Quarta", "Quinta", "Sexta"],
  "time_slots": ["07h00-07h50", "07h50-08h40", "08h40-09h30", "10h00-10h50", "10
     h50-11h40", "11h40-12h30", "14h00-14h50", "14h50-15h40", "15h40-16h30", "16
     h30-17h20", "17h20-18h10", "18h10-19h00"],

  "professor_preferences": [
        { "professor": "Ana Elisa", "preferred_days": ["Terça", "Quarta", "Quinta
           "] },
    { "professor": "André", "preferred_days": ["Quarta", "Quinta", "Sexta"] },
    { "professor": "Carlos Eduardo", "preferred_days": ["Sexta", "Segunda"] },
    { "professor": "Carolina", "preferred_days": ["Segunda", "Terça", "Quarta", "
       Quinta"] },
    { "professor": "Celso", "preferred_days": ["Terça", "Quarta", "Quinta"] },
    { "professor": "Cláudio", "preferred_days": ["Sexta", "Segunda", "Terça"] }
  ],

  "courses": [
    { "code": "Leitura e Produção de Textos", "workload": 2, "semester": 1, "
       professor": "Amanda", "type": "obrigatoria" },
    { "code": "Estruturas Discretas", "workload": 4, "semester": 2, "professor":
       "André", "prerequisites": ["Pré-Cálculo", "Introdução a Ciência da
       Computação"], "failure_rate": 0.6, "type": "obrigatoria" },
    { "code": "Cálculo a várias Variáveis", "workload": 5, "semester": 3, "
       professor": "Rafael Saraiva", "prerequisites": ["Cálculo a uma Variável"
       ], "failure_rate": 0.5, "type": "obrigatoria" },
     { "code": "Equações Diferenciais Ordinárias I", "workload": 4, "semester":
        4, "professor": "Carlos Eduardo", "prerequisites": ["Álgebra Linear", "C
        álculo a várias Variáveis"], "failure_rate": 0.5, "type": "obrigatoria"
        },
    { "code": "Cálculo Numérico", "workload": 4, "semester": 5, "professor": "
       Pedro Lara", "prerequisites": ["Álgebra Linear", "Introdução a Programaçã
       o", "Cálculo a várias Variáveis"], "failure_rate": 0.6, "type": "
       obrigatoria" },
    { "code": "Libras I", "workload": 2, "semester": 6, "professor": "Soraia", "
       type": "optativa"},
  ]
}
```

Apêndice B - Grade horária gerada

Figura 14: Grade horária do período 1.

Horário Acadêmico - Período 1

Horário	Segunda	Terça	Quarta	Quinta	Sexta
07h00-07h50	---	Geometria Analítica Ana Elisa	Leitura e Produção de Textos Amanda	---	---
07h50-08h40	Lógica para Computação João Thompson	Geometria Analítica Ana Elisa	Leitura e Produção de Textos Amanda	Pré-Cálculo João Thompson	Pré-Cálculo B Carlos Eduardo
08h40-09h30	Lógica para Computação João Thompson	Geometria Analítica Ana Elisa	Administração e Organização Empresarial Roberta	Pré-Cálculo João Thompson	Pré-Cálculo B Carlos Eduardo
10h00-10h50	Lógica para Computação João Thompson	Introdução a Ciência da Computação Luis Retondaro	Administração e Organização Empresarial Roberta	Pré-Cálculo João Thompson	Pré-Cálculo B Carlos Eduardo
10h50-11h40	Introdução a Engenharia Rafael Saraiva	Introdução a Ciência da Computação Luis Retondaro	Projeto de Interação Luis Retondaro	---	---
11h40-12h30	Introdução a Engenharia Rafael Saraiva	Introdução a Ciência da Computação Luis Retondaro	Projeto de Interação Luis Retondaro	---	---
14h00-14h50	Introdução a Ciência da Computação B Luis Retondaro	---	---	---	---
14h50-15h40	Introdução a Ciência da Computação B Luis Retondaro	---	---	---	---
15h40-16h30	Introdução a Ciência da Computação B Luis Retondaro	---	---	---	---

Legenda de Professores:
João Thompson · Rafael Saraiva · Luis Retondaro · Ana Elisa
Amanda · Roberta · Carlos Eduardo

Figura 15: Grade horária do período 2.

Horário Acadêmico - Período 2

Horário	Segunda	Terça	Quarta	Quinta	Sexta
07h00-07h50	---	Ética Profissional Carolina	Introdução a Economia Roberta	Estruturas Discretas André	Introdução a Programação João Thompson
07h50-08h40	Mecânica Clássica Felipe Mondaini	Ética Profissional Carolina	Introdução a Economia Roberta	Estruturas Discretas André	Introdução a Programação João Thompson
08h40-09h30	Mecânica Clássica Felipe Mondaini	Cálculo a uma Variável Rafael Saraiva	---	---	Álgebra Linear João Thompson
10h00-10h50	Mecânica Clássica Felipe Mondaini	Cálculo a uma Variável Rafael Saraiva	Cálculo a uma Variável Rafael Saraiva	---	Álgebra Linear João Thompson
10h50-11h40	Álgebra Linear João Thompson	Mecânica Clássica Felipe Mondaini	Cálculo a uma Variável Rafael Saraiva	Introdução a Programação João Thompson	Estruturas Discretas André
11h40-12h30	Álgebra Linear João Thompson	Mecânica Clássica Felipe Mondaini	Cálculo a uma Variável Rafael Saraiva	Introdução a Programação João Thompson	Estruturas Discretas André

Legenda de Professores:
Felipe Mondaini · João Thompson · Carolina · Rafael Saraiva
Roberta · André

Figura 16: Grade horária do período 3.

Horário Acadêmico - Período 3

Horário	Segunda	Terça	Quarta	Quinta	Sexta
07h00-07h50	Software Básico Pedro Lara	Cálculo a várias Variáveis Rafael Saraiva	Cálculo a várias Variáveis Rafael Saraiva	Humanidades e Ciências Sociais Celso	Modelagem de Dados Pedro Lugão
07h50-08h40	Software Básico Pedro Lara	Cálculo a várias Variáveis Rafael Saraiva	Cálculo a várias Variáveis Rafael Saraiva	Humanidades e Ciências Sociais Celso	Modelagem de Dados Pedro Lugão
08h40-09h30	Software Básico Pedro Lara	Termodinâmica Felipe Mondaini	Cálculo a várias Variáveis Rafael Saraiva	---	Algoritmos e Estruturas de Dados I Pedro Lugão
10h00-10h50	Software Básico Pedro Lara	Termodinâmica Felipe Mondaini	---	---	Algoritmos e Estruturas de Dados I Pedro Lugão
10h50-11h40	Termodinâmica Felipe Mondaini	---	Introdução a Engenharia Ambiental Roberta	Algoritmos e Estruturas de Dados I Pedro Lugão	Algoritmos e Estruturas de Dados I Pedro Lugão
11h40-12h30	Termodinâmica Felipe Mondaini	---	Introdução a Engenharia Ambiental Roberta	Algoritmos e Estruturas de Dados I Pedro Lugão	Algoritmos e Estruturas de Dados I Pedro Lugão

Legenda de Professores:

Pedro Lara | Felipe Mondaini | Rafael Saraiva | Roberta

Celso | Pedro Lugão

Figura 17: Grade horária do período 4.

Horário Acadêmico - Período 4

Horário	Segunda	Terça	Quarta	Quinta	Sexta
07h00-07h50	---	Arquitetura de Computadores Luis Tarrataca	Arquitetura de Computadores Luis Tarrataca	Eletromagnetismo Ana Barros	Banco de Dados Jurair
07h50-08h40	Eletromagnetismo Ana Barros	Arquitetura de Computadores Luis Tarrataca	Arquitetura de Computadores Luis Tarrataca	Eletromagnetismo Ana Barros	Banco de Dados Jurair
08h40-09h30	Eletromagnetismo Ana Barros	Algoritmos e Estruturas de Dados II Laura	Arquitetura de Computadores Luis Tarrataca	Redes de Computadores I Ana Elisa	Banco de Dados Jurair
10h00-10h50	Eletromagnetismo Ana Barros	Algoritmos e Estruturas de Dados II Laura	Arquitetura de Computadores Luis Tarrataca	Redes de Computadores I Ana Elisa	Banco de Dados Jurair
10h50-11h40	Equações Diferenciais Ordinárias I Carlos Eduardo	Algoritmos e Estruturas de Dados II Laura	Algoritmos e Estruturas de Dados II Laura	Redes de Computadores I Ana Elisa	Equações Diferenciais Ordinárias I Carlos Eduardo
11h40-12h30	Equações Diferenciais Ordinárias I Carlos Eduardo	Algoritmos e Estruturas de Dados II Laura	Algoritmos e Estruturas de Dados II Laura	Redes de Computadores I Ana Elisa	Equações Diferenciais Ordinárias I Carlos Eduardo
14h00-14h50	---	---	---	Tópicos Especiais em Computação Movél André	---
14h50-15h40	---	---	---	Tópicos Especiais em Computação Movél André	---
15h40-16h30	---	---	---	Tópicos Especiais em Computação Movél André	---
17h20-18h10	---	Equações Diferenciais Ordinárias I B Leandro	---	Equações Diferenciais Ordinárias I B Leandro	---
18h10-19h00	---	Equações Diferenciais Ordinárias I B Leandro	---	Equações Diferenciais Ordinárias I B Leandro	---

Legenda de Professores:

Ana Barros | Carlos Eduardo | Luis Tarrataca | Laura

Leandro | Ana Elisa | André | Jurair

Figura 18: Grade horária do período 5.

Horário Acadêmico - Período 5

Horário	Segunda	Terça	Quarta	Quinta	Sexta
07h00-07h50	---	Circuitos Lineares Cláudio	Cálculo Numérico Pedro Lara	Programação Orientada a Objetos Luis Claudio	Circuitos Lineares Cláudio
07h50-08h40	Probabilidade e Estatística Carlos Eduardo	Circuitos Lineares Cláudio	Cálculo Numérico Pedro Lara	Programação Orientada a Objetos Luis Claudio	Circuitos Lineares Cláudio
08h40-09h30	Probabilidade e Estatística Carlos Eduardo	Redes de Computadores II Dalbert	Cálculo Numérico Pedro Lara	Programação Orientada a Objetos Luis Claudio	Sistemas Operacionais André
10h00-10h50	Probabilidade e Estatística Carlos Eduardo	Redes de Computadores II Dalbert	Cálculo Numérico Pedro Lara	Programação Orientada a Objetos Luis Claudio	Sistemas Operacionais André
10h50-11h40	Engenharia de Software Luis Retondaro	Redes de Computadores II Dalbert	Redes de Computadores II Dalbert	Sistemas Operacionais André	Programação Orientada a Objetos Luis Claudio
11h40-12h30	Engenharia de Software Luis Retondaro	Redes de Computadores II Dalbert	Redes de Computadores II Dalbert	Sistemas Operacionais André	Programação Orientada a Objetos Luis Claudio
17h20-18h10	---	História e Filosofia da Ciência Marcilia	---	---	---
18h10-19h00	---	História e Filosofia da Ciência Marcilia	---	---	---

Legenda de Professores:

Carlos Eduardo · Luis Retondaro · Cláudio · Dalbert

Marcilia · Pedro Lara · Luis Claudio · André

Figura 19: Grade horária do período 6.

Horário Acadêmico - Período 6

Horário	Segunda	Terça	Quarta	Quinta	Sexta
07h00-07h50	Servidores de Redes Dalbert	Lab. de Circuitos Elétricos e Eletrônicos Cláudio	---	Sinais e Sistemas Marco Aurélio	Linguagens Formais e Autômatos Pedro Lugão
07h50-08h40	Servidores de Redes Dalbert	Lab. de Circuitos Elétricos e Eletrônicos Cláudio	---	Sinais e Sistemas Marco Aurélio	Linguagens Formais e Autômatos Pedro Lugão
08h40-09h30	Eletrônica Analógica Cláudio	Servidores de Redes Dalbert	Análise de Algoritmos Luis Tarrataca	Ondulatória e Física Moderna Rogério	Linguagens Formais e Autômatos Pedro Lugão
10h00-10h50	Eletrônica Analógica Cláudio	Servidores de Redes Dalbert	Análise de Algoritmos Luis Tarrataca	Ondulatória e Física Moderna Rogério	---
10h50-11h40	Sinais e Sistemas Marco Aurélio	Servidores de Redes Dalbert	Análise de Algoritmos Luis Tarrataca	Ondulatória e Física Moderna Rogério	Eletrônica Analógica Cláudio
11h40-12h30	Sinais e Sistemas Marco Aurélio	Servidores de Redes Dalbert	Análise de Algoritmos Luis Tarrataca	Ondulatória e Física Moderna Rogério	Eletrônica Analógica Cláudio
17h20-18h10	---	---	---	LIBRAS I - OPT Soraia	---
18h10-19h00	---	---	---	LIBRAS I - OPT Soraia	---

Legenda de Professores:

Dalbert · Cláudio · Marco Aurélio · Luis Tarrataca

Rogério · Soraia · Pedro Lugão

Figura 20: Grade horária do período 7.

Horário Acadêmico - Período 7

Horário	Segunda	Terça	Quarta	Quinta	Sexta
07h00-07h50	Processamento Digital de Sinais Marco Aurélio	---	Programação Linear Laura	---	---
07h50-08h40	Processamento Digital de Sinais Marco Aurélio	---	Programação Linear Laura	---	---
08h40-09h30	Processamento Digital de Sinais Marco Aurélio	---	Técnicas Digitais Cristiano	Técnicas Digitais Cristiano	---
10h00-10h50	Processamento Digital de Sinais Marco Aurélio	---	Técnicas Digitais Cristiano	Técnicas Digitais Cristiano	---
10h50-11h40	Metodologia Científica Cláudio	Programação Linear Laura	Técnicas Digitais Cristiano	---	---
11h40-12h30	Metodologia Científica Cláudio	Programação Linear Laura	Técnicas Digitais Cristiano	---	---
16h30-17h20	---	---	SEGURANÇA DE REDES DE COMPUTADORES II - OPT Dalbert	---	---
17h20-18h10	---	---	SEGURANÇA DE REDES DE COMPUTADORES II - OPT Dalbert	---	---
18h10-19h00	---	---	SEGURANÇA DE REDES DE COMPUTADORES II - OPT Dalbert	---	---

Legenda de Professores:
Marco Aurélio
Cláudio
Laura
Cristiano
Dalbert

Figura 21: Grade horária do período 8.

Horário Acadêmico - Período 8

Horário	Segunda	Terça	Quarta	Quinta	Sexta
07h00-07h50	---	---	Microcontroladores e Sistemas Embarcados Cristiano	---	---
07h50-08h40	Sistemas Inteligentes Rafael Saraiva	---	Microcontroladores e Sistemas Embarcados Cristiano	---	---
08h40-09h30	Sistemas Inteligentes Rafael Saraiva	Sistemas de Controle Cláudio	---	Sistemas Distribuídos André	Sistemas Distribuídos André
10h00-10h50	Sistemas Inteligentes Rafael Saraiva	Sistemas de Controle Cláudio	Computação Gráfica Luis Retondaro	Sistemas Distribuídos André	Sistemas Distribuídos André
10h50-11h40	---	Sistemas de Controle Cláudio	Computação Gráfica Luis Retondaro	Microcontroladores e Sistemas Embarcados Cristiano	---
11h40-12h30	---	Sistemas de Controle Cláudio	Computação Gráfica Luis Retondaro	Microcontroladores e Sistemas Embarcados Cristiano	---
16h30-17h20	---	---	---	NANOCIÊNCIA E NANOTECNOLOGIA - OPT Renata	---
17h20-18h10	---	---	---	NANOCIÊNCIA E NANOTECNOLOGIA - OPT Renata	---
18h10-19h00	---	---	---	NANOCIÊNCIA E NANOTECNOLOGIA - OPT Renata	---

Legenda de Professores:
Rafael Saraiva
Cláudio
Cristiano
Luis Retondaro
André
Renata

Figura 22: Grade horária do período 9.

Horário Acadêmico - Período 9

Horário	Segunda	Terça	Quarta	Quinta	Sexta
07h00-07h50	Trabalho de Conclusão de Curso I Luis Retondaro	Computação de Alto Desempenho Pedro Lara	---	---	---
07h50-08h40	Trabalho de Conclusão de Curso I Luis Retondaro	Computação de Alto Desempenho Pedro Lara	---	---	---
10h50-11h40	Computação de Alto Desempenho Pedro Lara	---	---	---	Estágio Supervisionado Jurair
11h40-12h30	Computação de Alto Desempenho Pedro Lara	---	---	---	Estágio Supervisionado Jurair
14h00-14h50	---	---	---	---	TÓPICOS ESPECIAIS EM IA - OPT Pedro Lugão
14h50-15h40	---	---	---	---	TÓPICOS ESPECIAIS EM IA - OPT Pedro Lugão
15h40-16h30	---	---	---	---	TÓPICOS ESPECIAIS EM IA - OPT Pedro Lugão
16h30-17h20	---	---	---	---	TÓPICOS ESPECIAIS EM IA - OPT Pedro Lugão
17h20-18h10	---	INTRO. CRIPTOGRAFIA - OPT Pedro Lara	---	---	---
18h10-19h00	---	INTRO. CRIPTOGRAFIA - OPT Pedro Lara	---	---	---

Legenda de Professores:
Luis Retondaro Pedro Lara Jurair Pedro Lugão

Figura 23: Grade horária do período 10.

Horário Acadêmico - Período 10

Horário	Segunda	Terça	Quarta	Quinta	Sexta
08h40-09h30	---	Trabalho de Conclusão de Curso II Laura	---	---	---
10h00-10h50	---	Trabalho de Conclusão de Curso II Laura	---	---	---
14h00-14h50	---	---	---	5G - COMUNICAÇÃO DE DADOS MÓVEIS - OPT Ana Elisa	---
14h50-15h40	---	---	---	5G - COMUNICAÇÃO DE DADOS MÓVEIS - OPT Ana Elisa	---
15h40-16h30	---	---	---	5G - COMUNICAÇÃO DE DADOS MÓVEIS - OPT Ana Elisa	---
16h30-17h20	---	---	---	INFRAESTRUTURA DE REDES PARA IOT - OPT Ana Elisa	---
17h20-18h10	---	---	---	INFRAESTRUTURA DE REDES PARA IOT - OPT Ana Elisa	---
18h10-19h00	---	---	---	INFRAESTRUTURA DE REDES PARA IOT - OPT Ana Elisa	---

Legenda de Professores:
Laura Ana Elisa